Logica Matematica

APPUNTI DEL CORSO DI LOGICA MATEMATICA
TENUTO DAL PROF. MARCELLO MAMINO

Diego Monaco
d.monaco2@studenti.unipi.it

Daria Pasqualetti
inserisci.mail@esempio.it

Università di Pisa

Anno Accademico 2024-25

Indice

Premessa

Queste dispense sono la quasi esatta trascrizione in LaTeX delle dispense del corso di Logica Matematica [**?**], tenuto dal prof. Marcello Mamino nell'anno accademico 2023-24 presso l'Università di Pisa.

Ringraziamenti

Antonio De Lucreziis.

Quest'opera è stata rilasciata con la licenza Creative Commons Attribuzione – Non commerciale 4.0 Internazionale. Per leggere una copia della licenza visita il sito web https://creativecommons.org/licenses/by-nc/4.0/deed.it.

Parte I

Semantica

Parte II

Sintassi

Parte III
Calcolabilità

§1 Teoria della calcolabilità

§1.1 Funzioni primitivo ricorsive

Vorremmo definire la classe delle funzioni implementabili in un calcolatore, tale classe è stata definita in vari modi, ad inizio '900 da Church, Turing, Post e altri, e si è poi visto che le varie definizioni date erano equivalenti, seppur magari non fosse quella l'intenzione originaria di chi le aveva date. Le principali definizioni che vediamo e che sono state date, sono:

1. tramite gli operatori di chiusura,
2. tramite un modello di calcolo (macchine di Turing),
3. tramite formule aritmetiche.

Definizione 1.1 (Funzioni primitive ricorsive - PRF)**.** Definiamo la classe delle **funzioni primitive ricorsive** a partire da alcune funzioni di base e chiudendo la classe sotto alcune operazioni di chiusura. Le funzioni di base vanno da $\mathbb{N}^k \to \mathbb{N}$ per qualche $k \in \mathbb{N}$ e sono:

- $0_k : \mathbb{N}^k \to \mathbb{N}$, per ogni $k \in \mathbb{N}$, tale che $0_k(x_1, \dots, x_k) = 0_k(\mathbf{x}) = 0$,
- $s : \mathbb{N} \to \mathbb{N}$, la funzione successore, $s(x) = x + 1$,
- $\pi_{k,i} : \mathbb{N}^k \to \mathbb{N}$, per ogni $k \in \mathbb{N}$ e $1 \leq i \leq k$, tale che $\pi_{k,i}(x_1, \dots, x_k) = x_i$.

Chiudiamo queste funzioni di base sotto le seguenti operazioni, siano $f : \mathbb{N}^a \to \mathbb{N}$ e $g_1, \dots, g_a : \mathbb{N}^b \to \mathbb{N}$ primitive ricorsive, allora:

$$\mathbb{N}^b \to \mathbb{N} : (x_1, \dots, x_b) \mapsto f(g_1(x_1, \dots, x_b), \dots, g_a(x_1, \dots, x_b))$$

è una funzione primitiva ricorsiva, cioè chiudiamo sotto composizione; siano $h : \mathbb{N}^k \to \mathbb{N}$ e $g : \mathbb{N}^{k+2} \to \mathbb{N}$ primitive ricorsive, allora:

$$f : \mathbb{N}^{k+1} \to \mathbb{N} : \begin{cases} f(0, y_1, \dots, y_k) = h(y_1, \dots, y_k) \\ f(x+1, y_1, \dots, y_k) = g(x, f(x, y_1, \dots, y_k), y_1, \dots, y_k) \end{cases}$$

è primitiva ricorsiva, cioè chiudiamo sotto ricorsione.

Osservazione 1.2 (Permutazione di primitive ricorsive) — Sia $f(x_1, \dots, x_k)$ primitiva ricorsiva, e sia $\sigma \in \mathrm{S}_k$ una permutazione, allora:

$$g(x_1, \dots, x_k) := f(x_{\sigma(1)}, \dots, x_{\sigma(k)})$$

è primitiva ricorsiva, infatti g si ottiene da f componendo con le proiezioni relative alla permutazione σ:

$$g(x_1, \dots, x_k) = f(\pi_{k,\sigma(1)}(x_1, \dots, x_k), \dots, \pi_{k,\sigma(k)}(x_1, \dots, x_k))$$

Anzi non è necessario che σ sia una permutazione, basta che sia una qualsiasi funzione da $\{1, \dots, k\}$ a $\{1, \dots, k\}$.

Notazione 1.3 (PR) — D'ora in avanti PR sarà l'abbreviazione di "primitivo ricorsivo/a".

Vediamo alcuni esempi di funzioni primitive ricorsive.

Esempio 1.4 (Funzioni costanti)
Qualsiasi funzione costante $c_k : \mathbb{N}^k \to \mathbb{N}$ è primitiva ricorsiva, infatti c_k si ottiene da 0_k componendo con la funzione successore s un numero finito di volte:

$$c_k(x_1, \dots, x_k) = s^k \circ 0_k(x_1, \dots, x_k)$$

Esempio 1.5 (Funzione somma)
La funzione somma $f(x, y) = x + y$ è primitiva ricorsiva, infatti è definita per ricorsione sulla x a partire da $h(y) := \pi_{1,1}(y)$ e $g(x, z, y) := s \circ \pi_{3,2}(x, z, y)$:

$$f(x, y) := \begin{cases} f(0, y) = h(y) = \pi_{1,1}(y) = y \\ f(x + 1, y) = g(x, f(x, y), y) = s \circ \pi_{3,2}(x, f(x, y), y) = s(f(x, y)) \end{cases}$$

Esempio 1.6 (Funzione prodotto)
La funzione prodotto $f(x, y) = x \cdot y$ è primitiva ricorsiva, infatti è definita per ricorsione sulla x a partire da $h(y) := 0_1(y)$ e $g(x, y, z) := \pi_{3,2}(x, y, z) + \pi_{3,3}(x, y, z)$ (che è PR perché è composizione di funzioni PR):

$$f(x, y) := \begin{cases} f(0, y) = h(y) = 0_1(y) = 0 \\ f(x + 1, y) = g(x, f(x, y), y) = f(x, y) + y \end{cases}$$

Esempio 1.7 (Funzione potenza)
La funzione potenza $f(x, y) = x^y$ è primitiva ricorsiva, infatti posso definire $f(x, y) := F(y, x)$, dove F è definita per ricorsione usando $h(x) := s \circ 0_1(x) = c_1$ e $g(a, b, c) = \pi_{3,2}(a, b, c) \cdot \pi_{3,3}(a, b, c)$:

$$F(y, x) := \begin{cases} F(0, x) = h(x) = c_1(x) = 1 \\ F(y + 1, x) = g(y, F(y, x), x) = F(y, x) \cdot x \end{cases}$$

Esempio 1.8 (Funzione predecessore)
Definiamo la funzione **predecessore** `pred` come:

$$\texttt{pred}(x) := \begin{cases} 0 & \text{se } x = 0 \\ x - 1 & \text{se } x > 0 \end{cases}$$

`pred` è primitiva ricorsiva, ed è definita per ricorsione da $h := 0_0$ e $g(x, z) := \pi_{2,1}(x, z)$:

$$\texttt{pred}(x) := \begin{cases} \texttt{pred}(0) = h(0) = 0_0(0) = 0 \\ \texttt{pred}(x + 1) = g(x, \texttt{pred}(x)) = \pi_{2,1}(x, \texttt{pred}(x)) = x \end{cases}$$

Esempio 1.9 (Sottrazione troncata)
Definiamo la funzione **sottrazione troncata** $\dot{-}$ come:

$$x \dot{-} y := \begin{cases} 0 & \text{se } x \leq y \\ x - y & \text{se } x > y \end{cases}$$

$\dot{-}$ è primitiva ricorsiva, infatti è definita per ricorsione sulla y a partire da $h(x) := \pi_{1,1}(x)$ e $g(a,b,c) := \texttt{pred} \circ \pi_{3,2}(a,b,c)$:

$$x \dot{-} y := \begin{cases} x \dot{-} 0 = h(x) = \pi_{1,1}(x) = x \\ x \dot{-} (y+1) = g(y, x \dot{-} y, x) = \texttt{pred} \circ \pi_{3,2}(y, x \dot{-} y, x) = \texttt{pred}(x \dot{-} y) \end{cases}$$

Notare come stia facendo ricorsione sull'altra variabile, e non su x, ma, come già visto, a meno di permutare, posso fare ricorsione su qualsiasi variabile, quindi è ininfluente.

Definizione 1.10 (Predicato)**.** $P \subseteq \mathbb{N}^k$ è un **predicato PR** se esiste una funzione $f : \mathbb{N}^k \to \mathbb{N}$ PR tale che:

$$f(\mathbf{x}) = \begin{cases} 0 & \text{se } \mathbf{x} \in P \\ 1 & \text{se } \mathbf{x} \notin P \end{cases}$$

Notazione 1.11 (Notazione per i predicati) — Eventualmente si confonderanno le notazioni $\mathbf{x} \in P$ e $P(\mathbf{x})$, intendendo con $P(\mathbf{x})$ la formula $f(\mathbf{x}) = 0$.

Esempio 1.12 (Il predicato `zerop`)
Un esempio di predicato è $\texttt{zerop} := \{0\}$, infatti è definito dalla funzione PR:

$$f(x) := 1 \dot{-} (1 \dot{-} x)$$

infatti, se $\mathbb{N} \ni x > 1$ allora $1 \dot{-} x = 0$ e $1 \dot{-} 0 = 1$, per cui $x \notin \texttt{zerop}$; se $x = 1$ vale ancora $f(1) = 1$, mentre se $x = 0$ si ha $1 \dot{-} 0 = 1$ e $1 \dot{-} 1 = 0$, quindi $x \in \texttt{zerop}$.

Osservazione 1.13 (Algebra di boole dei predicati PR) — I predicati PR sottoinsiemi dello stesso $\mathbb{N}^k$ formano un'algebra di boole.

Dimostrazione. Dati $P, Q \subseteq \mathbb{N}^k$ predicati PR devo verificare che i seguenti sono predicati PR: $P^{\complement}$, $P \cup Q$, $\mathbb{N}^k$[1].

$\boxed{\mathbb{N}^k}$ È un predicato prendendo la funzione PR data da 0_k.

$\boxed{P^{\complement}}$ Sia f la funzione PR che definisce P, allora $P^{\complement}$ è definito dalla funzione PR $1 \dot{-} f$.

$\boxed{P \cup Q}$ Siano f e g le funzioni PR che definiscono rispettivamente P e Q, allora $P \cup Q$ è definito dalla funzione PR $(1 \dot{-} ((1 \dot{-} f) + (1 \dot{-} g)))$, oppure da $f \cdot g$.

□

[1] In questo modo ottengo che gli insiemi formano un'algebra sulle parti in senso classico, che corrisponde ad una $\mathbb{F}_2$-algebra sulle parti, detta anche algebra booleana.

Esempio 1.14 (Insiemi di confronto)

Gli insiemi $\{x \leq y\}$, $\{x < y\}$, $\{x = y\}$, $\{x \neq y\}$[a] sono predicati PR. Per $\{x = y\}$ è sufficiente usare:

$$f(x, y) := 1 \dot{-} (1 \dot{-} (y \dot{-} x + x \dot{-} y))$$

Per $\{x \leq y\}$ si usa:

$$g(x, y) := 1 \dot{-} (1 \dot{-} (x \dot{-} y))$$

e $g(x, y) = 0$ se e solo se $x \leq y$. Mettendo assieme queste due cose si ottiene

$$\{x < y\} = \{x \leq y\} \cap \{x \neq y\}.$$

Simmetricamente per $\geq$ e $>$.

[a] Nel senso di $\{(x, y) \in \mathbb{N}^2 | \ldots\}$.

Osservazione 1.15 (Definizione per casi) — Se $P \subseteq \mathbb{N}^k$ è un predicato PR, e $f, g : \mathbb{N}^k \to \mathbb{N}$ sono funzioni PR, allora la funzione definita per casi da:

$$h(\mathbf{x}) := \begin{cases} f(\mathbf{x}) & \text{se } \mathbf{x} \in P \\ g(\mathbf{x}) & \text{se } \mathbf{x} \notin P \end{cases}$$

è una funzione PR, infatti posso scrivere h come: $h(\mathbf{x}) = f(\mathbf{x}) \cdot (1 \dot{-} P(\mathbf{x})) + g(\mathbf{x}) \cdot P(\mathbf{x})$, che è una composizione di funzioni PR, e quindi è PR.

Esempio 1.16 (Funzione modulo)

La funzione $x \bmod y = z \overset{\text{def}}{\iff} z < y \wedge y \mid x - z$, cioè il rappresentante di x modulo y, è una funzione PR poiché è definita per ricorsione e per casi:

$$x \bmod y := \begin{cases} 0 \bmod y = 0 \\ (x+1) \bmod y := \begin{cases} 0 & \text{se } x \bmod y + 1 = y \\ (x \bmod y) + 1 & \text{se } x \bmod y + 1 \neq y \end{cases} \end{cases}$$

cioè, visto che $x = qy + x \mod y$, allora $x + 1 = qy + (x \mod y + 1)$, per cui se $x \mod y + 1 = y$ allora $x + 1 = (q + 1)y + 0$, altrimenti $x + 1 = qy + (x \mod y + 1)$. Osservare inoltre che nella definizione per casi abbiamo usato il predicato PR $P(a, b) := \{a + 1 = b\}$, e la h per la ricorsione:

$$h(a, b, c) := \begin{cases} 0 & \text{se } a + 1 = b \\ a + 1 & \text{se } a + 1 \neq b \end{cases}$$

che è una funzione PR per definizione per casi.

Osservazione 1.17 (Somma dei valori di una funzione PR sotto un certo punto) — Sia $f : \mathbb{N}^{k+1} \to \mathbb{N}$ una funzione PR, allora la funzione $(y_1, \dots, y_k, x) \mapsto \sum_{i<x} f(y_1, \dots, y_k, i)$ è PR, infatti è definita per ricorsione a partire da $h(y_1, \dots, y_k) := 0$ e $g(a, b, c, d_1, \dots, d_k) := b + f(d_1, \dots, d_k, a)$:

$$\sum_{i<x} f(y_1, \dots, y_k, i) := \begin{cases} \sum_{i<0} f(y_1, \dots, y_k, i) & = h(y_1, \dots, y_k) = 0 \\ \sum_{i<x+1} f(y_1, \dots, y_k, i) & = \sum_{i<x} f(y_1, \dots, y_k, i) \\ & +f(y_1, \dots, y_k, x) \end{cases}$$

Esempio 1.18 (Funzione che conta gli elementi del predicato fino ad x)
Sia $P \subseteq \mathbb{N}^{k+1}$ un predicato PR, allora la funzione:

$$g : (y_1, \dots, y_k, x) \mapsto \#\{(y_1, \dots, y_k, i) \in P : i < x\}$$

è una funzione PR, infatti:

$$g(y_1, \dots, y_k, x) = \sum_{i<x} (1 \dot{-} P(y_1, \dots, y_k, i))$$

Esempio 1.19 (Appartenenza ad un altro predicato sotto un certo punto)
Definiamo il predicato $Q \subseteq \mathbb{N}^{k+1}$ come:

$$(y_1, \dots, y_k, x) \in Q \iff \exists i < x \; (y_1, \dots, y_k, i) \in P$$

con $P \subseteq \mathbb{N}^{k+1}$ predicato PR. Tale sottoinsieme è un predicato PR in quanto è possibile definirlo come:

$$Q(y_1, \dots, y_k, x) = \{g(y_1, \dots, y_k, x) > 0\}$$

dove $g(y_1, \dots, y_k, x) = \#\{(y_1, \dots, y_k, i) \in P : i < x\}$ è la funzione PR (serve affinché l'insieme di confronto sia un predicato PR) che conta gli elementi del predicato P sotto x.

Esempio 1.20 (Funzione quoziente intero)
Definiamo $x \,\texttt{div}\, y := \left\lfloor \frac{x}{y} \right\rfloor$, con la convenzione $x \,\texttt{div}\, 0 := 0$. La funzione $x \,\texttt{div}\, y$ è PR, ad esempio per ricorsione su x (con parametro y) e per casi:

$$\begin{cases} 0 \,\texttt{div}\, y = 0 \\ (x+1) \,\texttt{div}\, y := \begin{cases} 0 & \text{se } y = 0 \\ (x \,\texttt{div}\, y) + 1 & \text{se } y \neq 0 \text{ e } x \bmod y + 1 = y \\ x \,\texttt{div}\, y & \text{se } y \neq 0 \text{ e } x \bmod y + 1 \neq y \end{cases} \end{cases}$$

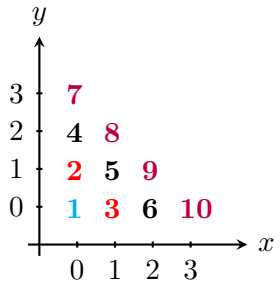
il predicato e la funzione usati per la ricorsione sono quasi gli stessi di quelli usati per definire il modulo, anche in questo caso vale che se $x = (x \,\texttt{div}\, y)y + r$, allora $x + 1 = (x \,\texttt{div}\, y)y + (r + 1)$, per cui se $r + 1 = y$ allora $x + 1 = (x \,\texttt{div}\, y + 1)y + 0$, altrimenti $x + 1 = (x \,\texttt{div}\, y)y + (r + 1)$.

§1.2 Codifica delle liste

Definizione 1.21 (Pairing function)**.** Definiamo la funzione **pairing function** (o funzione di accoppiamento):

$$\texttt{cons} : \mathbb{N}^2 \to \mathbb{N} : (x, y) \mapsto \frac{(x+y)(x+y+1)}{2} + x + 1$$

Tale funzione è biunivoca tra le coppie $(x, y) \in \mathbb{N}^2$ e i numeri naturali maggiori di 1 (voglio avere un null point, e quindi non voglio che $(0, 0)$ venga codificato da 0). La funzione di pairing ha il seguente grafico.



Definizione 1.22 (Funzioni inverse di `cons`)**.** Definiamo le funzioni $\texttt{car}, \texttt{cdr} : \mathbb{N} \to \mathbb{N}$ come le funzioni inverse di `cons`, ovvero:

$$\texttt{car}(\texttt{cons}(x, y)) := x$$
$$\texttt{cdr}(\texttt{cons}(x, y)) := y$$

che sono funzioni PR. Un modo per verificare che sono PR è osservare che possono essere scritte come:

$$\texttt{car}(z) = \sum_{i<z} \sum_{j<z} i \cdot (1 \dot{-} \{z = \texttt{cons}(i, j)\})$$
$$\texttt{cdr}(z) = \sum_{i<z} \sum_{j<z} j \cdot (1 \dot{-} \{z = \texttt{cons}(i, j)\})$$

cioè itero la somma finché non becco la coppia (i, j) tale che $z = \texttt{cons}(i, j)$, allora i due predicati danno 0 e quindi viene $1 \cdot i$ o $1 \cdot j$.

Esercizio 1.23 (Funzione di Ackermann)**.** Dimostrare che la funzione di Ackermann non è una funzione primitiva ricorsiva:[2]

$$A(m, n) := \begin{cases} A(0, n) = n + 1 \\ A(m + 1, 0) = A(m, 1) \\ A(m + 1, n + 1) = A(m, A(m + 1, n)) \end{cases}$$

[2] **Hint**: Un qualunque programma costituito solo da cicli `for` annidati un numero finito di volte, e senza ricorsione, non può catturare l'intera funzione.

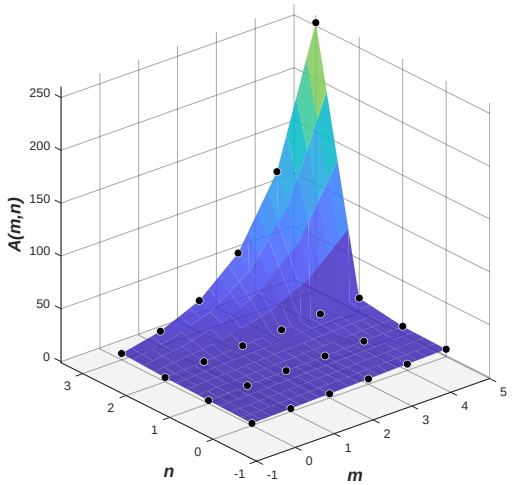


Figura 1: Plot della funzione di Ackermann

Osservazione 1.24 (Le sezioni $n \mapsto A(m,n)$ sono PR) — Per ogni $m \in \mathbb{N}$, la funzione $A_m : \mathbb{N} \to \mathbb{N}$ definita da $A_m(n) := A(m,n)$ è PR. Si dimostra per induzione su m: $A_0(n) = n+1$ è il successore, e se A_m è PR allora anche A_{m+1} è PR, perché è definita per ricorsione da:

$$A_{m+1}(0) = A_m(1), \qquad A_{m+1}(n+1) = A_m(A_{m+1}(n)).$$

L'idea che abbiamo voluto catturare, mediante le funzioni primitive ricorsive, è quella di una classe di funzioni che può essere implementata in un calcolatore solo per mezzo di cicli `for` - per cui sarà già a priori fissato il numero di loop, e che non usino né i costrutti della ricorsione (quindi una funzione non può chiamare se stessa) e né usi cicli potenzialmente illimitati, come i cicli `while`.

§1.3 Funzioni ricorsive

Definiamo inizialmente della notazione per rappresentare funzioni che non convergono.

Definizione 1.25 ($\mathbb{N}_\perp$)**.** Definiamo $\mathbb{N}_\perp := \mathbb{N} \cup \{\perp\}$.

Definizione 1.26 (Funzioni ricorsive)**.** Definiamo le **funzioni ricorsive** come le funzioni $f : \mathbb{N}_\perp^k \to \mathbb{N}_\perp$ a partire da alcune funzioni di base e chiudendo la classe sotto alcune operazioni di chiusura. Le funzioni di base, fissato $k \in \mathbb{N}$, vanno da $\mathbb{N}_\perp^k \to \mathbb{N}_\perp$ e sono:

$$\pi_{k,i}(x_1, \dots, x_k) = \begin{cases} x_i & \text{se } x_1, \dots, x_k \in \mathbb{N} \\ \perp & \text{altrimenti} \end{cases}$$

$$s(x) = \begin{cases} x+1 & \text{se } x \in \mathbb{N} \\ \perp & \text{se } x = \perp \end{cases}$$

$$0_k(x_1, \dots, x_k) = \begin{cases} 0 & \text{se } x_1, \dots, x_k \in \mathbb{N} \\ \perp & \text{altrimenti} \end{cases}$$

Partendo da queste funzioni faccio la chiusura per le operazioni di:

- composizione;
- ricorsione primitiva, con lo stesso schema visto per le funzioni PR, ma con l'aggiunta del caso $f(\perp, \dots) := \perp$;

- **minimizzazione**, ovvero se $f : \mathbb{N}^{k+1} \to \mathbb{N}_\perp$ è ricorsiva, allora la funzione:

$$\mu_k f(x_1, \dots, x_k) = a \in \mathbb{N} \overset{\text{def}}{\iff} f(x_1, \dots, x_k, a) = 0 \wedge \forall i < a \; f(x_1, \dots, x_k, i) \in \mathbb{N} \setminus \{0\}$$

  in particolare $f(x_1, \dots, x_k, i)$ non è $\perp$; se non esistesse un tale a allora poniamo $\mu_k f(x_1, \dots, x_k) := \perp$. Cioè $\mu_k f$ produce il valore a tale che $f(x_1, \dots, x_k, a) = 0$, e tale a è il minimo possibile - l'idea che si vuole catturare è quella della convergenza di un algoritmo, che si ferma al passo a, quando f produce 0, e se non si ferma allora $\mu_k f$ non converge e quindi la poniamo $\perp$.

Definizione 1.27 (Funzioni ricorsive totali)**.** Diciamo che $f : \mathbb{N}_\perp^k \to \mathbb{N}_\perp$ funzione ricorsiva è una **totale** se $\forall (x_1, \dots, x_k) \in \mathbb{N}^k \; f(x_1, \dots, x_k) \in \mathbb{N}$, cioè f non produce mai $\perp$.

Osservazione 1.28 (Input $\perp$) — Se f è ricorsiva ed uno degli input è $\perp$, allora f produce $\perp$:

$$f(x_1, \dots, x_{i-1}, \perp, x_{i+1}, \dots, x_k) = \perp$$

per ogni $i \in \{1, \dots, k\}$.

Dimostrazione. Si procede per induzione sulla costruzione di f. I casi delle 3 funzioni di base sono banali per la definizione stessa.

- Per la composizione, se $f = h \circ (g_1, \dots, g_m)$, e uno degli input di f è $\perp$, allora anche uno dei g_i è $\perp$, per cui h produce $\perp$ per ipotesi induttiva, e quindi f è $\perp$.
- Per la ricorsione primitiva, se f è definita per ricorsione da h e g, e uno degli input di f è $\perp$, allora anche uno degli input di h o di g è $\perp$, per cui h o g producono $\perp$ per ipotesi induttiva, e quindi anche f dà $\perp$.
- Per la minimizzazione, se $f = \mu_k f'$, e uno degli input di f è $\perp$, allora anche uno degli input di f' è $\perp$, per cui f' produce $\perp$ a prescindere dalla variabile per cui si minimizza, e non fornendo mai un valore in $\mathbb{N} \setminus \{0\}$, il minimo non esiste, e quindi anche f è $\perp$ per definizione.

$\square$

Fatto 1.29 (Inclusione tra le classi di funzioni ricorsive)**.** Valgono le seguenti inclusioni, tutte strette:

$$\{\text{funzioni PR}\} \subsetneq \{\text{funzioni ricorsive totali}\} \subsetneq \{\text{funzioni ricorsive}\}$$

Dimostrazione. I contenimenti sono tutti banali per le definizioni date, vediamo solo i controesempi per mostrare che sono stretti. Per il secondo contenimento si osserva che $\mu_0 s \equiv \perp$, perché l'equazione $s(x) = 0$ non ha soluzioni. Per il primo è sufficiente considerare la funzione di Ackermann. $\square$

§1.4 Macchine di Turing

Vogliamo caratterizzare le funzioni ricorsive come esattamente quelle che sono implementabili da una Macchina di Turing.

Definizione 1.30 (Macchina di Turing - TM)**.** Siano Σ un insieme finito di **simboli**, Q un insieme finito di **stati**, una **macchina di Turing** (**TM**) è il dato di Σ, Q, di una **funzione di transizione**:

$$\delta : Q \times \Sigma \to Q \times \Sigma \times \{\leftarrow, \downarrow, \rightarrow\}$$

dove l'ultimo insieme si rappresenta formalmente con $\{0, 1, 2\}$ (con la corrispondenza in questo esatto ordine), ma è più comodo rappresentarlo con le frecce. Una **configurazione** è il dato di (s, p, n) dove $s \in Q$ è lo stato della TM, $p \in \mathbb{N}$ la posizione, e $n \in \Sigma^*$[3] la stringa (o meglio tutto il nastro) in cui ci troviamo, i.e. $n \in \Sigma^{\mathbb{N}}$ che è quasi sempre 0, i.e. $\exists N \in \mathbb{N}\ \forall i > N\ n(i) = 0$.
Dico che la macchina di Turing M passa da una configurazione all'altra:

$$(s, p, n) \overset{M}{\to} (s', p', n')$$

se: $s' = \pi_{3,1} \circ \delta(s, n(p))$, $p' = (p + \pi_{3,3} \circ \delta(s, n(p))) \dot{-} 1$ cioè la posizione finale è quella iniziale più 1 se la freccia è $\rightarrow$, è quella iniziale se la freccia è $\downarrow$, e $p - 1$ se la freccia è $\leftarrow$ - dove la terza componente di δ è identificata col numero non con la freccia, e:

$$n'(i) = \begin{cases} n(i) & \text{se } i \neq p \\ \pi_{3,2} \circ \delta(s, n(p)) & \text{se } i = p \end{cases}$$

cioè scorrendo il nastro modifico solo la cella in cui mi trovo inizialmente, $n(p)$, lascio le altre uguali e mi sposto.

Definizione 1.31 (Chiusura transitiva)**.** Dico che $c = (s, p, n) \overset{M^*}{\to} c' = (s', p', n')$ è la **chiusura transitiva** di $c \overset{M}{\to} c'$, se esiste una sequenza finita di configurazioni $c_0, \ldots, c_k$ tale che:

$$c_0 \overset{M}{\to} c_1 \overset{M}{\to} \ldots \overset{M}{\to} c_k$$

con $c_0 = c$ e $c_k = c'$.

Esercizio 1.32 (TM che riconosce i palindromi)**.** Costruire una TM, ovvero la tabella di transizione, che riceva in input:

$$0110101010111 \ldots 10112000 \ldots$$

ed ha due stati terminali, 0 e $0'$, e termina in 0 se e solo se la stringa è palindroma. Operiamo nell'alfabeto $\Sigma = \{0, 1, 2\}$.

Esercizio 1.33 (Difficile)**.** Dimostra che una TM come nell'esercizio precedente, termina, nel caso peggiore in $O(\text{lunghezza stringa}^2)$ mosse - con O si intende $\leq kl^2 + o(l^2)$ per qualche $k > 0$.[4]

[3] Questa è la notazione di Kleene.

[4] **Hint**: L'algoritmo più sensato da implementare è: sostituire il primo simbolo con 2, poi spostarsi fino alla fine della stringa, sostituire l'ultimo simbolo con 2, e poi tornare indietro fino al primo simbolo, ripetere finché non si sostituisce tutta la stringa con dei 2, oppure finché non si trova un simbolo diverso da quello sostituito con 2, per cui se la stringa è lunga l, allora si fanno l passi per sostituire ciascun simbolo, e quindi l^2 passi in totale. Bisogna dimostrare che non si può fare meglio di così,

§1.5 Funzioni Turing-computabili

Definizione 1.34 (Turing-computabilità)**.** Sia $f : \mathbb{N}^k \to \mathbb{N}_\perp$[5] una funzione ricorsiva **parziale** - nel senso di non necessariamente totale, quindi ricorsiva generica, diciamo che f è **Turing-computabile** se esiste una TM M, con:

- $\Sigma \supseteq \{0, 1, \blacktriangleright\}$,
- $Q \supseteq \{0, 1\}$, con 0 **stop** e 1 **start**,
- $\delta(0, x) = (0, x, \downarrow)$, ovvero lo stato terminale rimane fermo, non cambia simbolo né si muove,

e tale per cui:

- se $f(x_1, \ldots, x_k) = y$, allora $\texttt{input}(x_1, \ldots, x_k) \overset{M^*}{\to} \texttt{output}(y)$, con $\texttt{input}(x_1, \ldots, x_k)$ e $\texttt{output}(y)$ due configurazioni,
- se $f(x_1, \ldots, x_k) = \perp$, allora $\texttt{input}(x_1, \ldots, x_k) \overset{M^*}{\not\to} (0, \ldots)$, ovvero non termina in uno stato 0, dove avverrebbe lo stop, i.e. se non c'è un output non termina mai.

Dove:

$$\texttt{input}(x_1, \ldots, x_k) := (1, 0, \blacktriangleright \underbrace{111111}_{x_1} 0 \underbrace{1111}_{x_2} 0 \ldots 0 \underbrace{111}_{x_k} 00 \ldots) \in Q \times \mathbb{N} \times \Sigma^*$$

dove il numero x_i è rappresentato da una sequenza di x_i simboli 1 (notazione unaria) - tanti quanti il valore di x_i, e tra un numero e l'altro c'è uno 0; analogamente:

$$\texttt{output}(y) := (0, 0, \blacktriangleright \underbrace{111 \ldots 1}_{y} 00 \ldots) \in Q \times \mathbb{N} \times \Sigma^*$$

dove y è rappresentato da una sequenza di y simboli 1, seguita da una sequenza infinita di 0.

Teorema 1.35 (Ricorsiva $\Rightarrow$ Turing-computabile)
Se $f : \mathbb{N}^k \to \mathbb{N}_\perp$ è ricorsiva parziale, allora f è Turing-computabile.

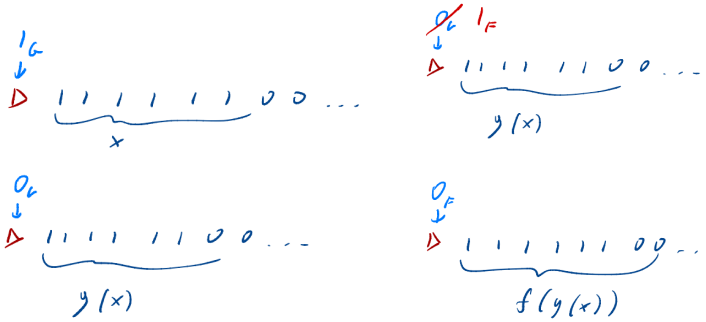
L'idea sarà far vedere che le funzioni di base sono Turing-computabili, e che le funzioni Turing computabili sono chiuse per operazioni di composizione, ricorsione primitiva e minimizzazione. Iniziamo con un caso particolare.

Esempio 1.36 (Composizione di funzioni Turing computabili)
Siano $f, g : \mathbb{N} \to \mathbb{N}_\perp$ funzioni Turing computabili, verifichiamo che $h(x) := f(g(x))$ è Turing computabile.

l'idea è che se la stringa è palindroma, la taglio alla metà esatta e posso ricostruire la metà sinistra usando come informazione tutta la successione di stati che ho ottenuto attraversando il confine. La TM deve attraversare il confine almeno $l/2$ volte, quindi decido di aggiungere un blocco di 0 tra la stringa e la metà riflessa, in modo che la TM non possa usare come informazione il numero di passi fatti, e quindi deve attraversare il confine almeno $l/2$ volte, e ogni volta deve fare almeno $l/2$ passi, per cui ottengo $O(l^2)$.

[5] Non mettiamo $\perp$ perché, avendo visto che se uno degli input è $\perp$ allora anche l'output è $\perp$, non ci interessa studiare funzioni di questo tipo.

Ciò che vorrei è, detta G la macchina associata a g e F quella associata a f, è far partire G dalla sequenza iniziale $\blacktriangleright$, farla terminare sempre a $\blacktriangleright$, e poi da qui far partire F e farla arrivare a $\blacktriangleright$ nello stato 0_F, facendo queste cose avrei di fatto costruito una macchina H che implementa h.



Posso costruire per bene una macchina H che implementa le operazioni fatte a mano.

Dimostrazione. Supponiamo per comodità che $\Sigma_G = \Sigma_F = \{0, 1, \blacktriangleright\} =: \Sigma_H$, definiamo $Q_H := Q_G \sqcup Q_F$, dove poniamo $1_H := 1_G$ e $0_H := 0_F$ - rispettivamente stato iniziale della macchina H e stato finale della macchina H. Per la funzione di transizione definiamo:

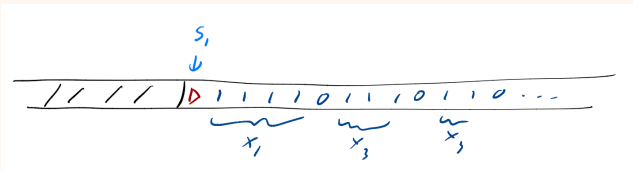
$$\delta_H : Q_H \times \Sigma_H \to Q_H \times \Sigma_H \times \{\leftarrow, \downarrow, \rightarrow\} : \delta_H(q, \sigma) := \begin{cases} \delta_G(q, \sigma) & \text{se } q \in Q_G \setminus \{0_G\} \\ \delta_F(1_F, \sigma) & \text{se } q = 0_G \\ \delta_F(q, \sigma) & \text{se } q \in Q_F \end{cases}$$

cioè se sono nello stato iniziale di G, mi comporto come se fossi in F - questo corrisponderebbe al nostro intervento manuale, altrimenti mi comporto come la macchina dello stato a cui appartengo. $\square$

Ora si pone il problema di generalizzare questo argomento alla composizione arbitraria. Costruiamo delle macchine che calcolino f, ma con qualche vincolo in più rispetto a quelli richiesti dalla definizione di Turing-computabilità, questa scelta è sia comoda che possibile, per cui la adottiamo. Data f vogliamo costruire la tabella delle funzioni di transizione della macchina che calcola f, lo facciamo a pezzi, cioè prendiamo dei sottoinsiemi degli stati e scriviamo dove vanno.

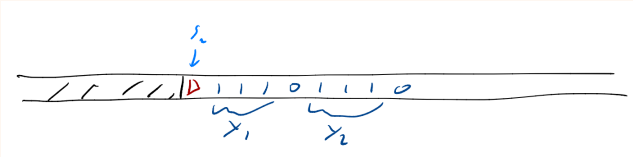
Setting: L'alfabeto per la costruzione di queste macchine sarà costituito solo dai tre simboli di base per la Turing-computabilità, ovvero $\Sigma = \{0, 1, \blacktriangleright\}$, per cui definiremo solo nuovi stati.

Notazione 1.37 (Notazione per descrivere il comportamento di pezzi di TM) — Con la notazione: $s_1 \to \texttt{macchina} \to s_2$ intendiamo che, preso un sottoinsieme di stati, dove s_1 è lo stato iniziale, che prendiamo sopra a $\blacktriangleright$, seguito da una configurazione



o non ritorna mai nello stato s_2, o ritorna, nello stato s_2, sul $\blacktriangleright$, in una nuova

configurazione. Assumiamo inoltre che le macchine che stiamo costruendo non vadano mai a sinistra di ▶.



Quindi $s_1 \to \texttt{macchina} \to s_2 \overset{\text{def}}{\iff}$ la `macchina` viene posta nello stato s_1 sopra a ▶, con la configurazione - sempre scritta in notazione unaria con gli 0 - che sarà indicata tra parentesi, allora, senza mai toccare ciò che sta a sinistra del triangolino, porta la macchina nello stato s_2 sopra a ▶, con una nuova configurazione - nella solita notazione - che sarà indicata tra parentesi, oppure non termina mai:

$$s_1 \to \texttt{macchina} \to s_2(s_1 \blacktriangleright x_1x_2\dots x_k\dots) = s_2 \blacktriangleright y_1y_2\dots$$

Quindi con questa notazione possiamo facilmente descrivere il comportamento di pezzi di macchine, e poi comporli per ottenere macchine più complesse.

Definizione 1.38 (Macchine elementari)**.** Definiamo le seguenti macchine elementari.

- $s_1 \to \texttt{push}_{n,i} s_2(s_1 \blacktriangleright x_1x_2\dots x_n) = s_2 \blacktriangleright x_1x_2\dots x_nx_i$, dove la nuova configurazione ha $n+1$ numeri, ha copiato l'i-esimo numero in fondo alla fila;
- $s_1 \to \texttt{rem}_{n,i} s_2(s_1 \blacktriangleright x_1x_2\dots x_n) = s_2 \blacktriangleright x_1x_2\dots x_{i-1}x_{i+1}\dots x_n$, dove la nuova configurazione ha $n-1$ numeri, ha rimosso l'i-esimo numero;
- $s_1 \to \texttt{exec}_{n,k} f s_2(s_1 \blacktriangleright x_1x_2\dots x_n) = s_2 \blacktriangleright x_1x_2\dots x_{n-k} f(x_{n-k+1},\dots,x_n)$, dove la nuova configurazione ha $n-k+1$ numeri, ha eseguito la funzione f sugli ultimi k numeri, e ha aggiunto il risultato in fondo alla fila;
- infine definiamo la macchina che controlla che l'i-esimo numero sia 0:

  $$s_1 \to \texttt{if}\, x_i = 0 \to s_2 \vee s_3(s_1 \blacktriangleright x_1x_2\dots x_n) = \begin{cases} s_2 \blacktriangleright x_1x_2\dots x_n & \text{se } x_i = 0 \\ s_3 \blacktriangleright x_1x_2\dots x_n & \text{se } x_i \neq 0 \end{cases}$$

  i simboli sono uguali, cambia solo lo stato in cui si ritrova la macchina su ▶, in base a se x_i è 0 o no.

Adesso il piano è: assumiamo di avere queste macchine elementari, e dimostriamo che la classe delle funzioni Turing-computabili, rappresentabili mediante macchine di Turing che hanno come alfabeto $\Sigma = \{0, 1, \blacktriangleright\}$ e che non vanno mai a sinistra di ▶, è chiusa per composizione, ricorsione primitiva e minimizzazione. Dobbiamo anche definire macchine di Turing che implementino le funzioni: successore, sottrazione troncata per 1, costante 0 e proiezione. Infine vediamo come implementare le macchine elementari e con questo abbiamo dimostrato che ogni funzione ricorsiva è Turing-computabile.

Dimostrazione. Vediamo che la classe delle funzioni Turing-computabili è chiusa per composizione, ricorsione primitiva e minimizzazione. In tutti i casi faremo la seguente cosa: assumiamo di avere le macchine elementari che implementano le operazioni di `push`, `rem` e `exec`, e vediamo come usarle per costruire macchine che implementano le funzioni ottenute per composizione, ricorsione primitiva e minimizzazione. In particolare le nuove macchine saranno ottenute unendo gli stati - in maniera disgiunta per evitare problemi - e incollando le funzioni di transizione (come nell'esempio precedente), ci limiteremo

pertanto a descrivere soltanto come sono fatte queste ultime, componendo quelle delle macchine elementari, e non descrivendo per intero le macchine che implementano le funzioni.

composizione

Caso particolare: consideriamo prima il caso $f : \mathbb{N}^2 \to \mathbb{N}_\perp$ e $g_1, g_2 : \mathbb{N}^2 \to \mathbb{N}_\perp$, e poi generalizziamo. Vogliamo verificare che $f(g_1(x_1, x_2), g_2(x_1, x_2))$ sia Turing computabile.
Siano F, G_1, G_2 le macchine che implementano f, g_1, g_2, il nostro obiettivo è costruire una macchina che parta da $\blacktriangleright x_1x_2$ e arrivi a $\blacktriangleright f(g_1(x_1, x_2), g_2(x_1, x_2))$ - cioè a $\blacktriangleright y$ con $y = f(g_1(x_1, x_2), g_2(x_1, x_2))$, come da definizione.
In primis vogliamo copiare gli input, altrimenti nel calcolo usando $\texttt{exec}_{2,2}g_1$ scompaiono e non posso riutilizzarli, quindi operiamo due passaggi di `push` per copiare x_1 e x_2 in fondo alla lista:

$$s_1 \to \texttt{push}_{2,1} \to \texttt{push}_{3,2} \to s_2$$

per cui $s_2 \blacktriangleright x_1x_2x_1x_2$. Ora, dallo stato s_2 applico $\texttt{exec}_{4,2}g_1$:

$$s_2 \to \texttt{exec}_{4,2}g_1 \to s_3$$

con $s_3 \blacktriangleright x_1x_2g_1(x_1, x_2)$. Ora copio di nuovo x_1 e x_2 in fondo alla lista (per usare `exec` devo usare i numeri in fondo quindi ho necessità di copiarli) e calcolo g_2:

$$s_3 \to \texttt{push}_{3,1} \to \texttt{push}_{4,2} \to \texttt{exec}_{5,2}g_2 \to s_4$$

con $s_4 \blacktriangleright x_1x_2g_1(x_1, x_2)g_2(x_1, x_2)$. A questo punto posso cancellare i primi due e calcolare f:

$$s_4 \to \texttt{rem}_{4,1} \to \texttt{rem}_{3,1} \to \texttt{exec}_{2,2}f \to s_5$$

con $s_5 \blacktriangleright f(g_1(x_1, x_2), g_2(x_1, x_2))$, infine vado da s_5 a 0 per stoppare la macchina.
Caso generale: se $f : \mathbb{N}^k \to \mathbb{N}_\perp$ e $g_1, \dots, g_k : \mathbb{N}^h \to \mathbb{N}_\perp$, vogliamo costruire una macchina che parta da $\blacktriangleright x_1 \dots x_h$ e arrivi a $\blacktriangleright f(g_1(x_1, \dots, x_h), \dots, g_k(x_1, \dots, x_h))$.
Iniziamo copiando tutti gli input in fondo alla lista:

$$^{6}1 \to \texttt{push}_{h,1} \to \texttt{push}_{h+1,2} \to \dots \to \texttt{push}_{2h-1,h} \to 2$$

con $2 \blacktriangleright x_1 \dots x_hx_1 \dots x_h$. Ora usiamo $\texttt{exec}_{2h,h}g_1$ per calcolare g_1:

$$2 \to \texttt{exec}_{2h,h}g_1 \to 3$$

con $3 \blacktriangleright x_1 \dots x_hg_1(x_1, \dots, x_h)$. Ripetendo queste operazioni k volte, e otteniamo lo stato $4 \blacktriangleright x_1 \dots x_hg_1(x_1, \dots, x_h) \dots g_k(x_1, \dots, x_h)$. A questo punto cancelliamo i primi h numeri e calcoliamo f:

$$4 \to \texttt{rem}_{h+k,1} \to \texttt{rem}_{h+k-1,1} \to \dots \to \texttt{rem}_{k+1,1} \to \texttt{exec}_{k,k}f \to 5$$

con $5 \blacktriangleright f(g_1(x_1, \dots, x_h), \dots, g_k(x_1, \dots, x_h))$, infine vado da 5 a 0 per stoppare la macchina. Restano i casi in cui $g_i(x_1, \dots, x_h) = \perp$ per qualche i, oppure $f(g_1(x_1, \dots, x_h), \dots, g_k(x_1, \dots, x_h)) = \perp$, in ambo i casi, per ipotesi, le macchine che implementano g_i o f non terminano, e quindi anche $\texttt{exec}g_i$ o $\texttt{exec}f$ non termina, per cui la macchina che abbiamo costruito non termina, e quindi è coerente con $f(g_1(x_1, \dots, x_h), \dots, g_k(x_1, \dots, x_h)) = \perp$.

[6] Per comodità, d'ora in poi chiameremo gli stati solo con i numeri.

ricorsione primitiva

Siano $h : \mathbb{N}^k \to \mathbb{N}_\perp$ e $g : \mathbb{N}^{k+2} \to \mathbb{N}_\perp$, Turing computabili, e vogliamo implementare $f : \mathbb{N}^{k+1} \to \mathbb{N}_\perp$, definita via ricorsione primitiva a partire da h e g:

$$f(x, y_1, \dots, y_k) := \begin{cases} f(0, y_1, \dots, y_k) := h(y_1, \dots, y_k) \\ f(x+1, y_1, \dots, y_k) := g(x, f(x, y_1, \dots, y_k), y_1, \dots, y_k) \end{cases}$$

Calcoliamo al contrario la ricorsione, e teniamoci un contatore in modo da sapere quando fermarci. Assumiamo, WLOG, di avere uno 0 alla fine della stringa di input, ovvero $1 \blacktriangleright xy_1 \dots y_k 0$, calcoliamo $f(0, y_1, \dots, y_k) = h(y_1, \dots, y_k)$:

$$1 \to \texttt{push}_{k+2,2} \to \texttt{push}_{k+3,3} \to \dots \to \texttt{push}_{2k+1,k+1} \to \texttt{exec}_{2k+2,k} h \to 2$$

con $2 \blacktriangleright xy_1 \dots y_k 0 h(y_1, \dots, y_k)$, ora controlliamo se siamo nel caso base o no:

$$2 \to \texttt{if}\, x = 0 \to \begin{cases} 3 & \text{se } x = 0 \\ 4 & \text{se } x \neq 0 \end{cases}$$

per cui $3 \blacktriangleright 0y_1 \dots y_k 0 h(y_1, \dots, y_k)$, e siamo arrivati in questo caso, è sufficiente applicare $k+2$ volte `rem` e poi andare a 0 per ottenere $0 \blacktriangleright f(0, y_1, \dots, y_k)$.
Dimostriamo preliminarmente, per induzione - il caso 0 è quello sopra, che, assumendo $x - t \neq 0$, che da $\blacktriangleright x - ty_1 \dots y_k t f(t, y_1, \dots, y_k)$, riusciamo a passare alla configurazione $\blacktriangleright x - (t+1) y_1 \dots y_k (t+1) f(t+1, y_1, \dots, y_k)$, in tal modo avremo dimostrato per induzione che se $x - t \neq 0$, allora possiamo passare alla configurazione con $t+1$; fatto questo si applica $\texttt{if}\, x - (t+1) = 0$ per verificare se siamo arrivati al caso base, ed in tal caso, a meno di cancellare i primi simboli abbiamo concluso, altrimenti si continua, in un altro stato, col passo ricorsivo. Osserviamo inoltre che, essendo $x \in \mathbb{N}$, si avrà necessariamente $x - t = 0$, per t sufficientemente grande e questo ci permette di passare dallo stato 4 alla terminazione.
Resta da verificare, per induzione, che si può passare da $\blacktriangleright x - ty_1 \dots y_k t f(t, y_1, \dots, y_k)$ a $\blacktriangleright x - (t+1) y_1 \dots y_k (t+1) f(t+1, y_1, \dots, y_k)$: ricopiamo la stringa e cambiamola nel mentre.

$$4 \to \texttt{push}_{k+3,1} \to \texttt{exec}_{k+4,1} \dot{-} 1 \to 5$$

con $5 \blacktriangleright x - ty_1 \dots y_k t f(t, y_1, \dots, y_k) x - (t+1)$, copiamo ora gli input $y_1, \dots, y_k$:

$$5 \to \texttt{push}_{k+4,2} \to \texttt{push}_{k+5,3} \to \dots \to \texttt{push}_{2k+3,k+1} \to 6$$

con $6 \blacktriangleright x - ty_1 \dots y_k t f(t, y_1, \dots, y_k) x - (t+1) y_1 \dots y_k$, ora ricopiamo di nuovo tutta la stringa (includendo nel mezzo $f(t, y_1, \dots, y_k)$ e t a cui applicare subito dopo $\texttt{exec} + 1$), cancelliamo quello che non serve e calcoliamo $\texttt{exec}g$. Cancellando con `rem` anche $f(t, y_1, \dots, y_k)$, otteniamo $\blacktriangleright x - (t+1) y_1 \dots y_k (t+1) f(t+1, y_1, \dots, y_k)$, e da qui si arriva allo stato terminale.
Rimane il caso in cui $h(y_1, \dots, y_k) = \perp$ o $g(x, f(x, y_1, \dots, y_k), y_1, \dots, y_k) = \perp$, in tal caso, per ipotesi, le macchine associate a h e g non terminano e di conseguenza non terminano $\texttt{exec}h$ o $\texttt{exec}g$, per cui non termina la macchina che implementa f, coerentemente col fatto che $f(x, y_1, \dots, y_k) = \perp$ come funzione ricorsiva.

minimizzazione

Sia $g : \mathbb{N}^{k+1} \to \mathbb{N}_\perp$ Turing computabile, e vogliamo implementare $f : \mathbb{N}^k \to \mathbb{N}_\perp$, definita via minimizzazione a partire da g:

$$f(y_1, \dots, y_k) := \mu_x g(x, y_1, \dots, y_k)$$

Partiamo da $1 \blacktriangleright 0y_1 \dots y_k$, dove usiamo lo 0 come base del contatore che cis serve. Dimostriamo che, partendo dallo stato $2 \blacktriangleright ty_1 \dots y_k$, con $t \in \mathbb{N}$, o si ottiene l'output

voluto, oppure si ritorna nello stesso stato ma con $t+1$. Copiamo in fondo i valori $t, y_1, \ldots, y_k$ e calcoliamo $g(t, y_1, \ldots, y_k)$:

$$2 \to \texttt{push}_{k+1,1} \to \texttt{push}_{k+2,2} \to \ldots \to \texttt{push}_{2k+1,k+1} \to \texttt{exec}_{2k+2,k+1} g \to 3$$

con $3 \blacktriangleright t y_1 \ldots y_k g(t, y_1, \ldots, y_k)$, ora controlliamo se $g(t, y_1, \ldots, y_k) = 0$:

$$3 \to \texttt{if}\, x_{k+2} = 0 \to \begin{cases} 4 & \text{se } g(t, y_1, \ldots, y_k) = 0 \\ 5 & \text{se } g(t, y_1, \ldots, y_k) \neq 0 \end{cases}$$

da 4 è sufficiente cancellare i simboli iniziali e andare a 0 per concludere. Nel ramo 5, invece, cancelliamo $g(t, y_1, \ldots, y_k)$ e torniamo a 2:

$$\begin{aligned} 5 \to \texttt{rem}_{k+2,k+2} \to \texttt{push}_{k+1,1} \to \texttt{exec}_{k+2,1} + 1 \to \texttt{push}_{k+2,2} \to \\ \to \texttt{push}_{k+3,3} \to \ldots \to \texttt{push}_{2k+1,k+1} \to \texttt{rem}_{2k+2,1} \to \texttt{rem}_{2k+1,1} \to \\ \to \ldots \to \texttt{rem}_{k+2,1} \to 2 \end{aligned}$$

Quindi la macchina termina esattamente quando trova il minimo a tale che $g(a, y_1, \ldots, y_k) = 0$. Se non esiste i tale che la computazione di $g(i, y_1, \ldots, y_k)$ termina, oppure se $g(i, y_1, \ldots, y_k) = \perp$ per qualche i prima di trovare un valore zero, allora la macchina non raggiunge mai lo stato 0, e questo è coerente con la definizione di quando si ha $f(y_1, \ldots, y_k) = \perp$ per minimizzazione.

□

Rimane ora da costruire le macchine elementari, e le macchine che implementano le funzioni s, 0_k, $\pi_{k,i}$ e $\dot{-}1$.

Osservazione 1.39 (TM che implementano $\pi_{k,i}$ e 0_k) — Osservare che $\pi_{k,i}$ è implementabile con una sequenza di `rem` che cancella tutti i numeri tranne l'i-esimo, ricordando che lavoriamo in notazione unaria si lascia solo il blocco di uni relativo all'i-esimo numero; 0_k è implementabile semplicemente prendendo la rappresentazione in notazione unaria dei k numeri, e cancellando tutto fino a lasciare solo $\blacktriangleright 00\ldots$, che rappresenta correttamente lo 0 in notazione unaria.

Esercizio 1.40. Costruire macchine di Turing che implementino le funzioni s e $\dot{-}1$.

Soluzione. Per implementare s, partendo da $1 \blacktriangleright 1\ldots1$, con x simboli 1, basta aggiungere un altro simbolo 1 alla fine della stringa:

$$1 \to \texttt{push}_{x,1} \to 0$$

Per implementare $\dot{-}1$, partendo da $1 \blacktriangleright x$, con x simboli 1, in primis controllo se $x = 0$, e poi decido se restituire 0 o fare la sottrazione:

$$1 \to \texttt{if}\, x = 0 \to \begin{cases} 2 & \text{se } x = 0 \\ 3 & \text{se } x \neq 0 \end{cases}$$

con $2 \blacktriangleright 0$ e $3 \blacktriangleright x$, ora riscrivo x in notazione unaria per comodità, $3 \blacktriangleright 1\ldots1$, con x simboli 1, e poi cancello un simbolo 1:

$$3 \to \texttt{rem}_{x,1} \to 0$$

In ambo i casi le nuove macchine saranno ottenute unendo gli stati - in maniera disgiunta - e unendo le funzioni di transizione in modo che lo stato di stop di quella applicata prima si comporti come se fosse lo stato iniziale di quella applicata dopo. □

Esercizio 1.41. Costruire macchine di Turing che implementino le funzioni `push`, `rem`, `exec` e `if` .

Soluzione. Per tutte le macchine elementari usiamo $\Sigma = \{0, 1, \blacktriangleright\}$ e $Q \supseteq \{0, 1\}$, ma che dipende da quale macchina stiamo costruendo.

$\boxed{\texttt{if}\, x_i = 0}$

Vogliamo costruire una macchina che vada da $1 \blacktriangleright x_1 \dots x_n$ a $2 \blacktriangleright x_1 \dots x_n$ se $x_i = 0$, e a $3 \blacktriangleright x_1 \dots x_n$ se $x_i \neq 0$. Quindi la tabella della funzione di transizione sarà composta da $4 + i$ righe che mi servono per arrivare a x_i e poi decidere se andare in 2 o 3, da cui $Q = \{0, 1, 2, 3, \alpha_1, \dots, \alpha_i\}$ e:

	$\blacktriangleright$	0	1
0	$\blacktriangleright\ 0 \downarrow$	$0\ 0 \leftarrow$	$1\ 0 \leftarrow$
1	$\blacktriangleright\ \alpha_1 \rightarrow$	$0\ \alpha_1 \rightarrow$	$1\ \alpha_1 \rightarrow$
$\alpha_j\ (1 \leq j \leq i-1)$	$\blacktriangleright\ \alpha_j \rightarrow$	$0\ \alpha_{j+1} \rightarrow$	$1\ \alpha_j \rightarrow$
α_i	$\blacktriangleright\ \alpha_i \rightarrow$	$0\ 2 \rightarrow$	$1\ 3 \rightarrow$
2	$\blacktriangleright\ 2 \downarrow$	$0\ 2 \leftarrow$	$1\ 2 \leftarrow$
3	$\blacktriangleright\ 3 \downarrow$	$0\ 3 \leftarrow$	$1\ 3 \leftarrow$

avendo definito Σ, Q e δ, abbiamo costruito la macchina che implementa `if` $x_i = 0$.

$\boxed{\texttt{rem}_{n,i}}$

Vogliamo costruire una macchina che parta da $1 \blacktriangleright x_1 \dots x_n$ e arrivi a $2 \blacktriangleright x_1 \dots x_{i-1} x_{i+1} \dots x_n$.

$\boxed{\texttt{exec}_{n,k} f}$

Vogliamo costruire una TM che parta da $s_1 \blacktriangleright x_1 \dots x_n$ e arrivi allo stato $s_2 \blacktriangleright x_1 \dots x_{n-k} f(x_{n-k+1}, \dots, x_n)$, supponiamo che la macchina che implementa f sia F - e sia già implementata. La configurazione iniziale corrisponde sul nastro a:

$$\blacktriangleright \underbrace{11\dots11}_{x_1} 0 \underbrace{11\dots11}_{x_2} 0 \dots 0 \underbrace{11\dots11}_{x_{n-k}} 0 \underbrace{11\dots11}_{x_{n-k+1}} 0 \dots 0 \underbrace{11\dots11}_{x_n} 00 \dots$$

in primis vogliamo che la macchina si sposti verso destra fino a raggiungere x_{n-k+1}; arrivati allo 0 prima di x_{n-k+1}, ci mettiamo $\blacktriangleright$, a questo punto si usa che la TM che implementa f non va mai a sinistra di $\blacktriangleright$. Fatto questo abbiamo $f(x_{n-k+1}, \dots, x_n)$, per cui bisogna tornare indietro e man mano cancellare fino a x_{n-k}, infine si continua a spostarsi a sinistra fino a raggiungere $\blacktriangleright$, e da qui si va a s_2, che è lo stato di output desiderato.

	$\blacktriangleright$	0	1
s_1	$\blacktriangleright\ s_1 \rightarrow$	$0\ \alpha_1 \rightarrow$	$1\ s_1 \rightarrow$
α_i	$\blacktriangleright\ \alpha_i \rightarrow$	$0\ \alpha_{i+1} \rightarrow$	$1\ \alpha_i \rightarrow$
α_{n-k-1}	$\blacktriangleright\ \alpha_{n-k-1} \rightarrow$	$\blacktriangleright\ 1_F \downarrow$	$1\ \alpha_{n-k-1} \rightarrow$
0_F	$0\ \beta \leftarrow$	$0\ 0_F \downarrow$	$1\ 0_F \downarrow$
β	$\blacktriangleright\ s_2 \downarrow$	$0\ \beta \leftarrow$	$1\ \beta \leftarrow$

Dove $\alpha_1, \dots, \alpha_{n-k-1}$ sono nuovi stati, e nella tabella vanno inclusi anche tutti gli stati di F con le relative transizioni. $\square$

Occupiamoci ora di dimostrare che se f è Turing-computabile, allora f è ricorsiva, vedremo una cosa più forte, ovvero che esiste una funzione ricorsiva che simula ogni funzione Turing-computabile.

Osservazione 1.42 — Ricordando le definizioni di `car` e `cdr`, come rispettivamente inversa sulla prima e sulla seconda componente di `cons`, essendo $\texttt{cons} : \mathbb{N}^2 \hookrightarrow \mathbb{N} \setminus \{0\}$, possiamo estendere le definizioni di `car` e `cdr` in 0, ponendo $\texttt{car}(0) = \texttt{cdr}(0) = 0$.

Definizione 1.43 (Codifica di una lista)**.** Definiamo $\langle a_1, \ldots, a_n \rangle$ come:

$$\texttt{cons}(a_1, \texttt{cons}(a_2, \ldots \texttt{cons}(a_n, 0) \ldots))$$

ovvero codifichiamo una sequenza di numeri in un numero utilizzando la funzione `cons` iterata. Alternativamente, se ne può dare una definizione ricorsiva:

- $\langle \rangle := 0$,
- $\langle a_1, \ldots, a_n \rangle := \texttt{cons}(a_1, \langle a_2, \ldots, a_n \rangle)$.

Osservazione 1.44 — Tale codifica è iniettiva, e si verifica per induzione + il fatto che `cons` è iniettiva + il fatto che $\texttt{cons}(a, b) \geq \max\{a, b\}$.

Esempio 1.45

La lista $\langle 7, 15, 0, 9 \rangle$ viene codificata come $\texttt{cons}(7, \texttt{cons}(15, \texttt{cons}(0, \texttt{cons}(9, 0))))$.

$\langle 7, 15, 0, 9 \rangle =$

$= \texttt{cons}(7, \texttt{cons}(15, \texttt{cons}(0, \texttt{cons}(9, 0))))$

$= (7, \cdot)\ (15, \cdot)\ (0, \cdot)\ (9, 0)$

Osservare che 0 alla fine lo abbiamo messo per indicare la fine della lista, secondo la definizione ricorsiva.

Definizione 1.46 (Funzioni PR sulle liste)**.** Definiamo le seguenti funzioni PR sulle liste:

- $\texttt{nth}(\langle a_0, \ldots, a_n \rangle, k) := a_k$ se $k \leq n$, e 0 altrimenti,
- $\texttt{length}(\langle a_1, \ldots, a_n \rangle) := n$,
- $\texttt{cat}(\langle a_1, \ldots, a_m \rangle, \langle b_1, \ldots, b_n \rangle) := \langle a_1, \ldots, a_m, b_1, \ldots, b_n \rangle$,
- $\texttt{head}(\langle a_1, \ldots, a_n \rangle, k) := \langle a_1, \ldots, a_k \rangle$ se $k \leq n$, e $\langle a_1, \ldots, a_n \rangle$ altrimenti,

Esercizio 1.47. Verificare che le funzioni `nth`, `length`, `cat` e `head` sono PR.

Soluzione. Vediamo solo il caso di `nth`. Definiamo la funzione f:

$$f(l, k) := \begin{cases} f(l, 0) = l \\ f(l, k+1) = \texttt{cdr}(f(l, k)) \end{cases}$$

di fatto $f(l,k) = \texttt{cdr}^k(l)$, i.e. $f(l,k)$ è la k-esima coda di l (osservare l'immagine nell'esempio sopra). Fissata la coda k-esima, prendendone la testa, ottengo proprio il k-esimo elemento della lista, per cui:

$$\texttt{nth}(l,k) = \texttt{car}(f(l,k)) = \texttt{car}(\texttt{cdr}^k(l))$$

Per cui `nth` è PR, essendo `car` e `cdr` PR, e la composizione di funzioni PR è PR.
Per `length` basterà osservare che $\texttt{length}(l) = \#\{i < l | \texttt{cdr}^i(l) \neq 0\}$, cioè il numero degli i per cui posso fare i-volte coda senza arrivare a 0, ora `cdr` è PR e la condizione $\neq$ è PR, così come $i < l$ (notare che dal fatto che `cons` sia maggiore di ciascuno degli elementi segue anche che la codifica di una lista è maggiore di ciascuno degli elementi della lista, per cui $i < l$ è equivalente, a posteriori, a $i < \texttt{length}(l)$). □

Lemma 1.48 (Ricorsione sul (de)corso dei valori)
Sia $f : \mathbb{N}^{n+1} \to \mathbb{N}$ una funzione e $g : \mathbb{N}^{n+1} \to \mathbb{N}$ PR, tali che:

$$f(x_1, \dots, x_n, y) = g(x_1, \dots, x_n, \langle f(\mathbf{x}, y-1), f(\mathbf{x}, y-2), \dots, f(\mathbf{x}, 0)\rangle)$$

per cui f è g sugli stessi valori, e sull'ultima componente è la codifica della lista di tutti i valori che assume con entrata $< y$. Se ho una tale f, allora f è PR.

Dimostrazione. Sia $h(\mathbf{x}, y) := \langle f(\mathbf{x}, y-1), \dots, f(\mathbf{x}, 0)\rangle$, se riesco a dimostrare che h è PR, allora:

$$f(\mathbf{x}, y) = g(\mathbf{x}, h(\mathbf{x}, y))$$

e f è PR, essendo g e h PR, e la composizione di funzioni PR è PR. Per cui basta dimostrare che h è PR, definiamola per ricorsione primitiva:

$$h(\mathbf{x}, y) := \begin{cases} h(\mathbf{x}, 0) = \langle\rangle = 0 \\ h(\mathbf{x}, y+1) = \texttt{cons}(g(\mathbf{x}, h(\mathbf{x}, y)), h(\mathbf{x}, y)) \end{cases}$$

per la definizione di codifica di una lista ho messo $h(\mathbf{x}, y)$ come coda di $g(\mathbf{x}, h(\mathbf{x}, y))$, che è $f(\mathbf{x}, y)$ per ipotesi. □

Esempio 1.49 (Successione di Fibonacci)
Dal lemma precedente è facile dimostrare che la successione di Fibonacci `fib` è PR, infatti:

$$\texttt{fib}(x) = g(\langle \texttt{fib}(x-1), \dots, \texttt{fib}(0)\rangle)$$

e possiamo definire g per casi su un predicato PR come segue:

$$g(l) := \begin{cases} 0 & \text{se } \texttt{length}(l) = 0 \\ 1 & \text{se } \texttt{length}(l) = 1 \\ \texttt{car}(l) + \texttt{car}(cdr(l)) & \text{altrimenti} \end{cases}$$

Osservazione 1.50 — Notare che tramite questo lemma è anche più facile verificare che `cat` e `head` sono PR, infatti:

- per `cat` si applica il lemma alla successione dei valori $\texttt{cat}(\texttt{cdr}^i(l), m)$: il passo

successivo si ottiene in modo PR da quello precedente usando `car`, `cdr` e `cons`;

- per `head` si applica il lemma alla successione $\texttt{head}(l, i)$ per $i \leq k$: il passo è definito in modo PR per casi, usando `nth` e `length` per decidere se aggiungere un nuovo elemento o lasciare invariato il valore precedente.

Teorema 1.51 (Funzione ricorsiva universale)
Esiste una funzione ricorsiva parziale $u : \mathbb{N}^2 \to \mathbb{N}_\perp$ tale che per ogni $f : \mathbb{N}^k \to \mathbb{N}_\perp$ Turing-computabile, esiste un numero $\ulcorner f \urcorner \in \mathbb{N}$, detto **codice** di f, tale che:

$$\forall x_1, \ldots, x_k \in \mathbb{N}\ f(x_1, \ldots, x_k) = u(\ulcorner f \urcorner, \langle x_1, \ldots, x_k \rangle)$$

ovvero esiste una funzione universale u che simula ogni funzione Turing-computabile, e le basta il codice della funzione e la codifica degli input.

Corollario 1.52 (Turing-computabilità $\Rightarrow$ Ricorsività)
Se $f : \mathbb{N}^k \to \mathbb{N}_\perp$ è Turing-computabile, allora f è ricorsiva.

Dimostrazione. La dimostrazione segue dal teorema osservando che:

$$f(x_1, \ldots, x_k) = u(\ulcorner f \urcorner, \texttt{cons}(x_1, \texttt{cons}(x_2, \ldots)))$$

e u è ricorsiva, $\ulcorner f \urcorner$ è costante, quindi è ricorsiva, `cons` è ricorsiva, e la composizione di funzioni ricorsive è ricorsiva. □

Corollario 1.53
$\forall n \in \mathbb{N}\ \exists u_n : \mathbb{N}_\perp^{n+1} \to \mathbb{N}_\perp$ ricorsiva, tale che, data $f : \mathbb{N}_\perp^n \to \mathbb{N}_\perp$ ricorsiva, esiste un codice $\ulcorner f \urcorner$ tale che:

$$\forall x_1, \ldots, x_n \in \mathbb{N}_\perp\ f(x_1, \ldots, x_n) = u_n(\ulcorner f \urcorner, x_1, \ldots, x_n)$$

Dimostrazione. È un indebolimento del corollario precedente, comunque fa comodo osservare che ogni funzione ricorsiva può essere simulata da una funzione universale. □

Osservazione 1.54 (Tesi di Church) — "La classe delle funzioni ricorsive coincide con la classe delle funzioni effettivamente calcolabili". Tale tesi non è un'affermazione matematica, ma è l'idea che esprime la filosofia dietro alle varie nozioni di calcolabilità che abbiamo dato - e anche non dato, e.g. il λ-calcolo, che è quella di descrivere tutte le funzioni che possono essere calcolate effettivamente nella realtà. La tesi di Church può essere espressa in maniera più forte o debole in base a quanto si richieda buona la corrispondenza tra computabilità pratica e modello teorico, noi ci siamo limitati al risultato semantico - ovvero a quale sia il valore effettivo, ma si può dare anche una corrispondenza tenendo conto della complessità, ed a seconda di quanto uno richiede che la corrispondenza sia precisa viene una tesi più forte o debole.

Passiamo alla dimostrazione del teorema.

Dimostrazione. Poiché f è Turing-computabile, esiste una macchina di Turing M con $\{0, 1, \blacktriangleright\} \supseteq \Sigma, \{0, 1\} \supseteq Q \subseteq \mathbb{N}$, chiamiamo **configurazione** $= \langle \text{stato}, \text{posizione}, \text{nastro} \rangle$, dove nastro $= \langle a_0, a_1, \ldots, a_n \rangle$, che rappresenta il nastro $a_0 a_1 \ldots a_n 00 \ldots$, chiaramente questa codifica non è univoca perché posso aggiungere 0 in fondo ed ottenere la stessa configurazione, ma questo non è un problema.
Ci serve anche una codifica della funzione di transizione $\delta : Q \times \Sigma \to Q \times \Sigma \times \{\leftarrow, \downarrow, \rightarrow\}$, posso codificarla con un numero, $\ulcorner \delta \urcorner$, tale che:

$$\texttt{nth}(\ulcorner \delta \urcorner, \langle \text{stato}, \text{simbolo} \rangle) = \langle \text{stato'}, \text{simbolo'}, \text{spostamento} \rangle$$

tale che $\delta(\text{stato}, \text{simbolo}) = (\text{stato'}, \text{simbolo'}, \text{spostamento})$, ovvero $\ulcorner \delta \urcorner$ è il numero la cui posizione nella lista $\langle \text{stato}, \text{simbolo} \rangle$ contiene la lista $\langle \text{stato'}, \text{simbolo'}, \text{spostamento} \rangle$ che rappresenta l'output di δ su quell'input - e non è iniettiva.

Lemma 1.55

Esiste una funzione PR $\texttt{step} : \mathbb{N}^2 \to \mathbb{N}$ tale che per ogni codifica di configurazione c:

$$c \overset{M}{\to} \texttt{step}(\ulcorner \delta \urcorner, c)$$

cioè `step` calcola la codifica della configurazione successiva a c per la macchina M.

Dimostrazione. Per la codifica fatta:

- stato $= \texttt{nth}(c, 0)$,
- posizione $= \texttt{nth}(c, 1)$,
- nastro $= \texttt{nth}(c, 2)$.

per cui:

$$\textit{nastro} = \langle \underbrace{a_1, \ldots, a_{p-1}}_{\texttt{head}(\text{nastro,posizione})=:\alpha}, \underbrace{a_p}_{\texttt{nth}(\text{nastro,posizione}):=\beta}, \underbrace{a_{p+1}, \ldots, a_n}_{\texttt{cdr}^{\text{posizione}+1}(\text{nastro})=:\gamma} \rangle$$

ora la nuova configurazione dipende da $\texttt{nth}(\ulcorner \delta \urcorner, \langle \text{stato}, \beta \rangle)$, ovvero $\texttt{nth}(\ulcorner \delta \urcorner, \texttt{cons}(\text{stato}, \texttt{cons}(\beta, 0)))$ che chiamo `update`, per cui il nuovo nastro che ottengo è:

$$\text{nastro'} = \texttt{cat}(\alpha, \texttt{cons}(\texttt{nth}(\texttt{update}, 1), \gamma))$$

il nuovo stato è:

$$\langle \texttt{nth}(\texttt{update}, 0),\ \text{posizione} + \texttt{nth}(\texttt{update}, 2) \dot{-} 1,\ \text{nastro'} \rangle.$$

Uniamo queste cose e troviamo la funzione esplicita. □

Vogliamo descrivere una funzione che dice "in un certo numero di step, da un certo input arrivo ad un certo output", per cui definiamo:

$$\texttt{codifica_input}(\langle x_1, \ldots, x_k \rangle) := \langle 1, 0, \texttt{cons}(0, \texttt{nastro}(\langle x_1, \ldots, x_k \rangle)) \rangle$$

dove $\texttt{nastro}(\langle x_1, \ldots, x_k \rangle)$ è $\texttt{cat}(\texttt{unario}(x_1), \texttt{cons}(0, \texttt{nastro}(\langle x_2, \ldots, x_k \rangle)))$. Ora vogliamo dire che c'è una funzione che fa un certo numero di step alla volta, ovvero che la funzione:

$$(n, c, \ulcorner \delta \urcorner) \mapsto \texttt{step}^n(\ulcorner \delta \urcorner, c)$$

è PR, e questo è banale da verificare per induzione su n. Vogliamo procurarci, tramite minimizzazione, l'n al quale la macchina termina, consideriamo:

$$\mathtt{n}(c, \ulcorner\delta\urcorner) := \mu n \left(\mathtt{nth}(\mathtt{step}^n(\ulcorner\delta\urcorner, c), 0) = 0\right)$$

i.e il numero di passi in cui la macchina di Turing δ si ferma su input c. Consideriamo:

$$\mathtt{conta_uni}\left(\mathtt{nth}\left(\mathtt{step}^{\mathtt{n}(\mathtt{codifica_input}(\mathbf{x}), \ulcorner\delta\urcorner)}(\ulcorner\delta\urcorner, \mathtt{codifica_input}(\mathbf{x})), 2\right)\right)$$

che è il numero di 1 che la macchina di Turing δ lascia in output dopo essersi fermata su input $\mathtt{codifica_input}(\mathbf{x})$, e quindi è proprio $f(\mathbf{x})$. Per il conteggio usiamo la funzione PR:

$$\mathtt{conta_uni}(x) := \begin{cases} 0 & \text{se } \mathtt{car}(x) = 0 \\ 1 + \mathtt{conta_uni}(\mathtt{cdr}(x)) & \text{se } \mathtt{car}(x) = 1 \end{cases}$$

□

Corollario 1.56 (Forma normale per le funzioni ricorsive)
$\forall k \in \mathbb{N}$ esiste $T_k : \mathbb{N}^{k+2} \to \mathbb{N}$ predicato PR, e una funzione $U : \mathbb{N} \to \mathbb{N}$ PR, tali che $\forall f : \mathbb{N}^k \to \mathbb{N}_\perp$ ricorsiva (parziale) vale:

$$\forall x_1, \ldots, x_k \; f(x_1, \ldots, x_k) \neq \perp \iff \exists n \; T_k(x_1, \ldots, x_k, \ulcorner f \urcorner, n) = 0$$

i.e. $x_1, \ldots, x_k$ sono nel dominio di f se e solo se esiste un n tale che il predicato PR T_k applicato all'input di f, al codice di f ed a n è 0, e se questo avviene allora:

$$f(x_1, \ldots, x_k) = U(\mu_n(T_k(x_1, \ldots, x_k, \ulcorner f \urcorner, n) = 0))$$

i.e. se $x_1, \ldots, x_k$ sono nel dominio di f, allora $f(x_1, \ldots, x_k)$ è proprio U applicato al minimo n per cui T_k è 0.

§1.6 Teoremi di punto fisso e s-m-n

Teorema 1.57 ($s - m - n$)
Esiste una funzione ricorsiva totale (in realtà PR):

$$s_n^m : \mathbb{N}^{m+1} \to \mathbb{N}$$

tale che per ogni $\alpha, x_1, \ldots, x_m, y_1, \ldots, y_n \in \mathbb{N}$ vale:

$$u_{n+m}(\alpha, \langle x_1, \ldots, x_m, y_1, \ldots, y_n \rangle) = u_n(s_n^m(\alpha, x_1, \ldots, x_m), \langle y_1, \ldots, y_n \rangle)$$

ovvero s_n^m è una funzione che, dato il codice di una funzione calcolabile, e alcuni degli input e li fissa, restituisce il codice di una funzione che simula la funzione originale con quegli input fissati.

Esempio 1.58 (Somma PR)
Sia $f : \mathbb{N}^2 \to \mathbb{N} : (x, y) \mapsto x + y$, allora esiste $s_1^1 : \mathbb{N}^2 \to \mathbb{N}$ tale che:

$$u_2(\ulcorner f \urcorner, x, y) = u_1(s_1^1(\ulcorner f \urcorner, x), y)$$

ovvero s_1^1 è una funzione che, dato il codice di una funzione calcolabile f di due variabili, e un input x, restituisce il codice di una funzione che simula f con il primo input fissato a x, e quindi che prende un solo input y e restituisce $f(x, y)$. Ad esempio per $x = 7$, ho $\ulcorner g \urcorner := s_1^1(\ulcorner f \urcorner, 7)$, e $g(y) = f(7, y) = 7 + y$, e vale:

$$g(y) = u_1(\ulcorner g \urcorner, y) = u_1(s_1^1(\ulcorner f \urcorner, 7), y) = u_2(\ulcorner f \urcorner, 7, y) = f(7, y) = 7 + y$$

Teorema 1.59 ((Secondo) teorema di punto fisso)
Sia $h : \mathbb{N} \to \mathbb{N}$ una funzione ricorsiva totale, e sia $n \in \mathbb{N}$, allora esiste α tale che:

$$\forall x_1, \ldots, x_n \; u_n(\alpha, x_1, \ldots, x_n) = u_n(h(\alpha), x_1, \ldots, x_n)$$

i.e. per ogni funzione ricorsiva totale h, esiste un codice α - che codifica una funzione - tale che la funzione codificata da α è uguale alla funzione codificata da $h(\alpha)$, ovvero h ha un punto fisso per quanto riguarda la codifica di funzioni ricorsive.

Esempio 1.60 (Successione di Fibonacci PR - v.2)
Consideriamo la classica funzione di Fibonacci $f : \mathbb{N} \to \mathbb{N}$ definita da:

$$f(n) := \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \\ f(n \dot{-} 1) + f(n \dot{-} 2) & \text{altrimenti} \end{cases}$$

che sappiamo essere già PR per il lemma sulla ricorsione sul decorso dei valori, ma vediamo come dimostrarlo usando il secondo teorema di punto fisso. Definiamo la funzione:

$$g(\alpha, n) := \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \\ u_1(\alpha, n \dot{-} 1) + u_1(\alpha, n \dot{-} 2) & \text{altrimenti} \end{cases}$$

che questa sia una funzione ricorsiva (non necessariamente totale) è ovvio, a questo punto definisco:

$$h(\alpha) := s_1^1(\ulcorner g \urcorner, \alpha)$$

cioè la funzione che prende un codice α e restituisce il codice di una funzione che simula $g(\alpha, \cdot)$, "$\alpha \mapsto \ulcorner g(\alpha, \cdot) \urcorner$", sia α un punto fisso di h, tale che $u_1(\alpha, x) = u_1(h(\alpha), x)$, e dico che $f(x) := u_1(\alpha, x)$, per cui la successione di Fibonacci coincide con una funzione ricorsiva. Vediamo che f coincide con la funzione che ha codice α:

$$\begin{aligned} u_1(\alpha, x) &= u_1(h(\alpha), x) && \text{(punto fisso)} \\ &= u_1(s_1^1(\ulcorner g \urcorner, \alpha), x) && \text{(definizione di } h\text{)} \\ &= u_2(\ulcorner g \urcorner, \alpha, x) && \text{(definizione di } s_1^1\text{)} \\ &= g(\alpha, x) && \text{(definizione di funzione universale)} \end{aligned}$$

e, per la definizione di g, si ha che $u_1(\alpha, x)$ soddisfa esattamente la definizione ricorsiva di f. Se dimostro nella metateoria che c'è una sola funzione che soddisfa la definizione ricorsiva di f - ed in ZFC è il teorema di ricorsione numerabile, allora f coincide con $u_1(\alpha, x)$, e quindi f è ricorsiva.[a]

[a] Morale: la nozione di funzione ricorsiva che aveva Gödel è quella giusta.

Dimostriamo il teorema $s - m - n$[7].

Dimostrazione. Iniziamo con l'osservare che basta fare il caso $m = 1$, infatti se riesco a dimostrare che esiste s_n^1, allora posso iterare m volte s_n^1 per ottenere s_n^m.
Quindi vogliamo solo dimostrare che:

$$\forall \alpha \; \forall x_1, \mathbf{y} \; u_{n+1}(\alpha, x_1, \mathbf{y}) = u_n(s_n^1(\alpha, x_1), \mathbf{y})$$

L'idea è modificare la TM che implementa la funzione codificata da α in modo che, quando riceve un input x_1, invece di usarlo come input, lo usi per modificare la propria codifica, e poi si comporti come se avesse ricevuto un input x_1 - e quindi simuli la funzione originale con x_1 fissato.
Per fare questo, partendo da un input $1 \blacktriangleright y_1 \dots y_n$ devo descrivere una macchina che mi porti nello stato $\blacktriangleright x y_1 \dots y_n$, fatto questo è sufficiente attaccare questa macchina a

[7] Una sorta di half-evaluation per gli informatici.

quella che implementa la funzione codificata da α per concludere. Descriviamo quindi la macchina che ci serve in termini delle transizioni, usando le macchine elementari:

$$1 \to \underbrace{\texttt{exec}_{n+1,n+1}s \to \texttt{exec}_{n+1,n+1}s \to \ldots \to \texttt{exec}_{n+1,n+1}s}_{x \text{ volte}} \to 2$$

con $2 \blacktriangleright y_1 \ldots y_n x$, ho usato s sullo 0 immediatamente a destra di y_n; a questo punto non resta che copiare tutto davanti e rimuovere tutto quello che c'è prima:

$$2 \to \texttt{push}_{n+1,1} \to \texttt{push}_{n+2,2} \to \ldots \to \texttt{push}_{n+n,n} \to 3$$

con $3 \blacktriangleright y_1 \ldots y_n x y_1 \ldots y_n$, e infine:

$$3 \to \texttt{rem}_{2n+1,1} \to \texttt{rem}_{2n,1} \to \ldots \to \texttt{rem}_{n+2,1} \to 4$$

con $4 \blacktriangleright x y_1 \ldots y_n$, e da qui si attacca la macchina che implementa la funzione codificata da α, i.e. $4 \to 1_\alpha$. È necessario verificare che la funzione che abbiamo descritto è PR, e si conclude. □

Esercizio 1.61 (Effetto collaterale del teorema $s-m-n$)**.** Dimostra che c'è $k : \mathbb{N} \to \mathbb{N}$ ricorsiva totale tale che:

$$\forall n \in \mathbb{N}\ n = u_0(k(n))$$

ovvero c'è una funzione ricorsiva totale che, dato un numero n, restituisce un codice di una funzione costante sempre n.

Soluzione. Applichiamo il teorema $s-m-n$ alla funzione $\mathrm{id}_\mathbb{N} : \mathbb{N} \to \mathbb{N}$, che è PR - è la proiezione, e definisco:

$$k(n) := s_0^1(\ulcorner \mathrm{id}_\mathbb{N} \urcorner, n)$$

per cui:

$$u_0(k(n)) = u_0(s_0^1(\ulcorner \mathrm{id}_\mathbb{N} \urcorner, n)) = u_1(\ulcorner \mathrm{id}_\mathbb{N} \urcorner, n) = n$$

e quindi k soddisfa la richiesta. □

Esercizio 1.62. Data $f : \mathbb{N}_\perp \to \mathbb{N}_\perp$ ricorsiva, dimostra che esiste $k_f : \mathbb{N} \to \mathbb{N}$ ricorsiva totale tale che:

$$\forall n\ f(n) \in \mathbb{N} \implies f(n) = u_0(k_f(n))$$

Soluzione. Una possibilità è definire $k_f(n) := k(f(n))$, dove k è la funzione del precedente esercizio - ma questa non è necessariamente ricorsiva totale, quindi andrebbe verificato. Altrimenti, possiamo definire $k_f(n) := s_0^1(\ulcorner f \urcorner, n)$ - questo è sicuramente ricorsivo totale, perché s_0^1 è addirittura PR. □

Vediamo ora la dimostrazione del (secondo) teorema di punto fisso.

Dimostrazione. Abbiamo h fissata e cerchiamo una α tale che $u_n(\alpha, \mathbf{x}) = u_n(h(\alpha), \mathbf{x})$ per ogni $\mathbf{x}$. Definiamo

$$g(x, \mathbf{y}) := u_n(h(s_n^1(x, x)), \mathbf{y})$$

e

$$F(x) := s_n^1(\ulcorner g \urcorner, x).$$

Poniamo ora $\alpha := F(\ulcorner g \urcorner) = s_n^1(\ulcorner g \urcorner, \ulcorner g \urcorner)$. Allora:

$$\begin{aligned} u_n(\alpha, \mathbf{y}) &= u_n(s_n^1(\ulcorner g \urcorner, \ulcorner g \urcorner), \mathbf{y}) \\ &= u_{n+1}(\ulcorner g \urcorner, \ulcorner g \urcorner, \mathbf{y}) \end{aligned}$$

$$
\begin{aligned}
&= g(\ulcorner g \urcorner, \mathbf{y}) \\
&= u_n(h(s_n^1(\ulcorner g \urcorner, \ulcorner g \urcorner)), \mathbf{y}) \\
&= u_n(h(\alpha), \mathbf{y}).
\end{aligned}
$$

Quindi α è il punto fisso richiesto. □

§1.7 Insiemi decidibili, semidecidibili e ricorsivamente enumerabili

Definizione 1.63 (Insiemi decidibili, semidecidibili e ricorsivamente enumerabili)**.** Sia $R \subseteq \mathbb{N}^k$, allora R è:

- **semidecidibile** se esiste $f : \mathbb{N}^k \to \mathbb{N}_\perp$ ricorsiva parziale tale che $R = \{\mathbf{x} \in \mathbb{N}^k | f(\mathbf{x}) = 0\}$,
- **decidibile** se esiste f ricorsiva totale $f : \mathbb{N}^k \to \mathbb{N}$ tale che $R = \{\mathbf{x} \in \mathbb{N}^k | f(\mathbf{x}) = 0\}$,
- **ricorsivamente enumerabile** se esiste $f : \mathbb{N} \to \mathbb{N}^k$ ricorsiva totale tale che $R = \{f(n) | n \in \mathbb{N}\}$.

Proposizione 1.64 (Equivalenza delle nozioni di decidibilità)
Sia $R \subseteq \mathbb{N}^k$, allora le seguenti sono equivalenti:

1. R è semidecidibile,
2. R è ricorsivamente enumerabile,
3. R è dominio di una funzione ricorsiva parziale, $R = \{\mathbf{x} \in \mathbb{N}^k | \exists y\ f(\mathbf{x}) = y \neq \perp\}$.

Dimostrazione. Vediamo 1. $\implies$ 3., 3. $\implies$ 2. e 2. $\implies$ 1..

$\boxed{1. \Rightarrow 3.}$ Se R è semidecidibile, esiste $f : \mathbb{N}^k \to \mathbb{N}_\perp$ ricorsiva parziale tale che

$$R = \{\mathbf{x} \in \mathbb{N}^k : f(\mathbf{x}) = 0\}.$$

Definiamo $\tilde{f} : \mathbb{N}^{k+1} \to \mathbb{N}_\perp$ con

$$\tilde{f}(\mathbf{x}, n) := f(\mathbf{x}),$$

e poi

$$h(\mathbf{x}) := \mu n\, \tilde{f}(\mathbf{x}, n).$$

Se $\mathbf{x} \in R$, allora $f(\mathbf{x}) = 0$ e quindi $h(\mathbf{x}) = 0$. Se invece $\mathbf{x} \notin R$, allora $f(\mathbf{x}) \in \mathbb{N} \setminus \{0\}$ oppure $f(\mathbf{x}) = \perp$, quindi non esiste alcun n con $\tilde{f}(\mathbf{x}, n) = 0$ e dunque $h(\mathbf{x}) = \perp$. Quindi

$$R = \{\mathbf{x} \in \mathbb{N}^k : h(\mathbf{x}) \neq \perp\},$$

cioè R è il dominio di una funzione ricorsiva parziale.

$\boxed{3. \Rightarrow 2.}$ Se R è dominio di una funzione ricorsiva parziale f, per la forma normale esistono T, U PR tali che

$$f(\mathbf{x}) = U\big(\mu_y(T_{k+1}(\ulcorner f \urcorner, \mathbf{x}, y) = 0)\big).$$

Dunque

$$\mathbf{x} \in R \iff \exists y\, T_{k+1}(\ulcorner f \urcorner, \mathbf{x}, y) = 0.$$

Enumerando in diagonale le coppie $(\mathbf{x}, y)$ e stampando $\mathbf{x}$ quando la condizione PR vale, si ottiene una funzione totale ricorsiva $e : \mathbb{N} \to \mathbb{N}^k$ con immagine esattamente R (eventualmente con ripetizioni). Quindi R è ricorsivamente enumerabile.

$\boxed{2. \Rightarrow 1.}$ Se R è ricorsivamente enumerabile, esiste $e : \mathbb{N} \to \mathbb{N}^k$ ricorsiva totale con $R = \{e(n) : n \in \mathbb{N}\}$. Sia $\chi_=$ un predicato PR su $\mathbb{N}^k \times \mathbb{N}^k$ che vale 0 esattamente quando i due k-uple sono uguali. Definiamo

$$m(\mathbf{x}) := \mu n \left(\chi_=(e(n), \mathbf{x}) = 0\right).$$

Allora $m(\mathbf{x})$ converge se e solo se $\mathbf{x} \in R$. Componendo con la costante zero, $g := 0_1 \circ m$ è ricorsiva parziale e soddisfa

$$\mathbf{x} \in R \iff g(\mathbf{x}) = 0,$$

quindi R è semidecidibile.

□

Proposizione 1.65

R è decidibile se e solo se R e $R^{\complement}$ sono semidecidibili.

Osservazione 1.66 (Il complementare di un insieme decidibile è decidibile) — Se R è decidibile, allora $R^{\complement}$ è decidibile, infatti se $f : \mathbb{N}^k \to \mathbb{N}$ è ricorsiva totale tale che $R = \{f(\mathbf{x}) = 0\}$, allora $R^{\complement} = \{1 \dot{-} f(\mathbf{x}) = 0\}$, e quindi è decidibile, perché la funzione ottenuta dalla ricorsione è ancora ricorsiva totale.

Dimostrazione. È ovvio che se R è decidibile, allora R e $R^{\complement}$ sono semidecidibili. Per il viceversa, se R e $R^{\complement}$ sono semidecidibili, per la proposizione precedente sono enumerabili, quindi esistono $f, g : \mathbb{N} \to \mathbb{N}^k$ ricorsive totali tali che:

$$R = \{f(n) | n \in \mathbb{N}\} \quad \text{e} \quad R^{\complement} = \{g(n) | n \in \mathbb{N}\}$$

a questo punto posso definire $h : \mathbb{N} \to \mathbb{N}^k$ come:

$$h(n) := \begin{cases} f(n \,\texttt{div}\, 2) & \text{se } n \bmod 2 = 0 \\ g(n \,\texttt{div}\, 2) & \text{altrimenti} \end{cases}$$

in pratica h alterna le due enumerazioni. Sia ancora $\chi_=$ il predicato PR di uguaglianza su $\mathbb{N}^k$ e definiamo

$$m(\mathbf{x}) := \mu n \left(\chi_=(h(n), \mathbf{x}) = 0\right).$$

Per ogni $\mathbf{x}$ esiste almeno un tale n (perché $\mathbf{x} \in R$ oppure $\mathbf{x} \in R^{\complement}$), quindi m è totale. Inoltre

$$\mathbf{x} \in R \iff m(\mathbf{x}) \bmod 2 = 0.$$

Quindi il test di appartenenza a R è ricorsivo totale, cioè R è decidibile. □

Vediamo un insieme non decidibile.

Proposizione 1.67 (Halting problem)
Sia $H_1 := \{(\alpha, x) \in \mathbb{N}^2 | u_1(\alpha, x) \neq \bot\}$, ovvero l'insieme di codice-input per cui la macchina universale in dimensione 1 si ferma. Vale che H_1 non è decidibile.

Dimostrazione. Se per assurdo H_1 fosse decidibile, allora la funzione definita per casi:

$$f(\alpha) := \begin{cases} u_1(\alpha, \alpha) + 1 & \text{se } (\alpha, \alpha) \in H_1 \\ 0 & \text{altrimenti} \end{cases}$$

sarebbe ricorsiva totale: dato α, prima decido se $(\alpha, \alpha) \in H_1$; se no restituisco 0, se sì allora simulo $u_1(\alpha, \alpha)$ fino all'arresto (che esiste proprio perché siamo in H_1) e poi aggiungo 1. Usando $\alpha = \ulcorner f \urcorner$ si danno due casi:

- se $(\ulcorner f \urcorner, \ulcorner f \urcorner) \in H_1$, cioè il calcolatore termina su input $\ulcorner f \urcorner$, e quindi $f(\ulcorner f \urcorner) = u_1(\ulcorner f \urcorner, \ulcorner f \urcorner) + 1$, ma $u_1(\ulcorner f \urcorner, \ulcorner f \urcorner) = f(\ulcorner f \urcorner)$, per cui $f(\ulcorner f \urcorner) = f(\ulcorner f \urcorner) + 1$, assurdo;
- oppure se $(\ulcorner f \urcorner, \ulcorner f \urcorner) \notin H_1$, cioè il calcolatore non termina su input $\ulcorner f \urcorner$, e quindi $f(\ulcorner f \urcorner) = 0$, ma $u_1(\ulcorner f \urcorner, \ulcorner f \urcorner) = f(\ulcorner f \urcorner) = 0$ e non $\bot$, per cui $(\ulcorner f \urcorner, \ulcorner f \urcorner) \in H_1$, assurdo.

$\square$

Esercizio 1.68 (Halting problem generalizzato)**.** Dimostrare che H_n non è decidibile.

Esercizio 1.69. Dimostrare che H_n è semidecidibile.

Esercizio 1.70 (Insiemi non semidecidibili)**.** Dimostrare che l'insieme $TOT := \{\alpha | n \mapsto u_1(\alpha, n)$ è totale$\}$, ovvero l'insieme dei codici delle funzioni ricorsive totali, non è semidecidibile e che $TOT^{\complement}$ non è semidecidibile.[8]: Esibisco una f ricorsiva totale tale che, $\alpha \in \mathbb{N}^2 \setminus H_1 \iff f(\alpha) \in TOT$.

Esercizio 1.71. Esistono A, B semidecidibili, con $A \cap B = \varnothing$, tali che $\neg \exists C$ decidibile con $A \subseteq C$ e $B \subseteq C^{\complement}$. Ovvero non esistono due insiemi semidecidibili disgiunti che possono essere separati da un insieme decidibile.

[8] **Hint**

§2 Gerarchie aritmetiche

§2.1 Insiemi aritmetici

Definizione 2.1 (Insiemi aritmetici)**.** Diciamo che $R \subseteq \mathbb{N}^k$ è **aritmetico** se è definibile in $(\mathbb{N}; +, \cdot, 0, s)$, ovvero:

$$R = \{(x_1, \dots, x_k) \in \mathbb{N}^k | (\mathbb{N}; +, \cdot, 0, s) \models \varphi(x_1, \dots, x_k)\}$$

i.e. aritmetico $= L_{\text{arit}}$-definibile.

Definizione 2.2 (Classi di complessità aritmetica)**.** Diciamo che una formula aritmetica φ è **di classe**:

- $\Delta^0_0 = \Sigma^0_0 = \Pi^0_0$ se si ottiene dagli L_{arit}-termini, per chiusura sotto: $\neg$, $\wedge$, $\vee$, $\forall x \leq y$, $\exists x \leq y$, dove le ultime due sono abbreviazioni per formule della forma $\forall x\,(x \leq y \to \dots)$ e $\exists x\,(x \leq y \wedge \dots)$, con $x < y \overset{\text{def}}{=} \exists t\,(x + t = y)$, per cui possiamo applicare solo **quantificatori limitati** - i.e. quantificatori che quantificano su un intervallo finito, e quindi non possono essere usati per esprimere proprietà di insiemi infiniti;
- Σ^0_{n+1} se $\varphi = \exists x_1 \dots \exists x_k \psi$ con ψ di classe Π^0_n;
- Π^0_{n+1} se $\varphi = \forall x_1 \dots \forall x_k \psi$ con ψ di classe Σ^0_n.

Ovvero il pedice conta il numero di alternanze di blocchi di quantificatori dello stesso tipo, la lettera greca indica se il blocco di quantificatori più esterno è esistenziale o universale.

$$\begin{tikzcd}
 & \Delta^0_0 = \Sigma^0_0 = \Pi^0_0 \arrow[dl, dash] \arrow[dr, dash] & \\
\Sigma^0_1 = \exists\Pi^0_0 \arrow[d, dash] \arrow[drr, dash] & & \Pi^0_1 = \forall\Sigma^0_0 \arrow[d, dash] \arrow[dll, dash] \\
\Sigma^0_2 = \exists\Pi^0_1 \arrow[d, dash] & & \Pi^0_2 = \forall\Sigma^0_1 \arrow[d, dash] \\
\vdots & & \vdots
\end{tikzcd}$$

Esempio 2.3 (p è primo)

La formula $\mathsf{prime}(p) := \forall n \leq p\ (n = 1 \vee n = p \vee \neg\exists m \leq p\ m \cdot n = p)$, che esprime che p è un numero primo, è di classe Π^0_1, perché è una formula con un blocco di quantificatori universali all'esterno, e all'interno c'è una formula con quantificatori limitati, che è di classe Δ^0_0.

Esempio 2.4

Sia $p(x, y)$ polinomio fissato. Il predicato binario

$$R = \{(x, y) \in \mathbb{N}^2 : p(x, y) = 0\}$$

è di classe Δ_0^0, perché la formula $p(x, y) = 0$ è senza quantificatori. Invece il predicato unario

$$S = \{x \in \mathbb{N} : \exists y\, p(x, y) = 0\}$$

è di classe Σ_1^0.

Definizione 2.5 (Classe di un sottoinsieme di $\mathbb{N}^k$)**.** $R \subseteq \mathbb{N}^k$ è di classe $\square$ se esiste una L_{arit}-formula φ di classe $\square$ tale che:

$$R = \{(x_1, \dots, x_k) \in \mathbb{N}^k : \mathbb{N} \models \varphi(x_1, \dots, x_k)\}$$

Osservazione 2.6 — Ogni insieme aritmetico appartiene alla gerarchia aritmetica, è sufficiente portarla in forma normale premessa.

Definizione 2.7 (Classe Δ_n^0)**.** $R \subseteq \mathbb{N}^k$ è di classe Δ_n^0 se R è sia un insieme di classe Σ_n^0 che di classe Π_n^0.

Osservazione 2.8 (Chiusura delle classi aritmetiche) — Valgono le seguenti.

- Σ_n^0 è chiusa sotto: $\wedge$, $\vee$, $\exists$, $\forall y \leq x$,
- Π_n^0 è chiusa sotto: $\wedge$, $\vee$, $\forall$, $\exists y \leq x$,
- $X \subseteq \mathbb{N}^k$ è di classe Σ_n^0 se e solo se $X^{\complement}$ è di classe Π_n^0.

Dimostrazione. La terza è immediata dalla definizione del complementare. Inoltre, poiché la negazione scambia Σ_n^0 e Π_n^0, la prima affermazione è duale della seconda: basta dimostrare la prima. Siano dunque φ, ψ formule di classe Σ_n^0.

$\wedge, \vee$ Portando entrambe in forma premessa, posso accodare i blocchi esistenziali esterni e lasciare dentro una congiunzione/disgiunzione di formule di classe Π_{n-1}^0; resto quindi in Σ_n^0.

$\exists$ È banale: aggiungere un blocco esistenziale esterno non aumenta la complessità oltre Σ_n^0.

$\forall y \leq x$ Scrivo $\varphi(x, y)$ in forma $\exists z_1 \dots \exists z_r\, \theta(x, y, \mathbf{z})$ con $\theta \in \Pi_{n-1}^0$. Allora

$$\forall y \leq x\, \varphi(x, y) \iff \exists b\, \forall y \leq x\, \exists z_1 \leq b \dots \exists z_r \leq b\, \theta(x, y, \mathbf{z}).$$

L'equivalenza vale perché i testimoni dei quantificatori esistenziali sono cercati su un insieme finito di valori di y. I quantificatori limitati non aumentano il livello, quindi la formula interna è di classe Π_n^0; con l'esistenziale esterno torno in Σ_n^0.

$\square$

Corollario 2.9 (Chiusura di Δ_n^0)
Δ_n^0 è stabile per: $\wedge$, $\vee$, $\neg$, $\forall y \leq x$, $\exists y \leq x$.

Teorema 2.10 (Semidecidibilità in termini della gerarchia aritmetica)
$S \subseteq \mathbb{N}^k$ è semidecidibile se e solo se S è di classe Σ_1^0.

Corollario 2.11 (Decidibilità in termini della gerarchia aritmetica)
$S \subseteq \mathbb{N}^k$ è decidibile se e solo se S è di classe Δ_1^0.

Vediamo subito la dimostrazione del corollario.

Dimostrazione. Sappiamo che $S \subseteq \mathbb{N}^k$ è decidibile se e solo se S e $S^{\complement}$ sono semidecidibili, che equivale per il teorema a dire che S è di classe Σ_1^0 e $S^{\complement}$ è di classe Σ_1^0. Ora $S^{\complement}$ è di classe Σ_1^0 se e solo se S è di classe Π_1^0, per cui S è sia di classe Σ_1^0 che di classe Π_1^0, e quindi è di classe Δ_1^0, e viceversa.
Abbiamo usato che se un insieme è di classe Σ_n^0 allora il suo complementare è di classe Π_n^0, ciò deriva dal fatto che la negazione inverte i quantificatori, e dalla definizione di classe di un sottoinsieme di $\mathbb{N}^k$. $\square$

Esempio 2.12 (Insiemi diofantei)
L'insieme $\{(x_1, \ldots, x_k) \in \mathbb{N}^k : p(x_1, \ldots, x_k) = 0\}$ è di classe Δ_0^0, infatti la L_{arit}-formula $p(x_1, \ldots, x_k) = 0$ è senza quantificatori. Quando verificheremo che Δ_0^0 è un sottoinsieme dei predicati PR avremo:

$$\{\text{polinomi}\} \subseteq \Delta_0^0 \subseteq \{\text{predicati PR}\} \subseteq \{\text{decidibili}\} \subseteq \{\text{semidecidibili}\} = \Sigma_1^0$$

ora $\exists\Sigma_1^0 = \Sigma_1^0$, e se faccio questa cosa con Δ_0^0 ottengo $\exists\Delta_0^0 = \Sigma_1^0$, e quindi tutta la catena diventa Σ_1^0, per cui:

$$\{(x_1, \ldots, x_k) | \exists y_1 \ldots \exists y_h \; p(x_1, \ldots, x_k, y_1, \ldots, y_h) = 0\} \subseteq \Sigma_1^0$$

questi sono gli insiemi diofantei (parametrici). Quindi, in base a quanto dimostrato sopra, vale l'inclusione:

$$\{\text{insiemi diofantei parametrici}\} \subseteq \Sigma_1^0$$

in particolare, ogni insieme diofanteo parametrico è semidecidibile (equivalentemente, ricorsivamente enumerabile).

Ci servono dei lemmi per dimostrare il teorema.

Lemma 2.13
Sia $f : \mathbb{N}^k \to \mathbb{N}_\perp$ ricorsiva parziale, allora esiste $\varphi(x_1, \ldots, x_k, y)$ di classe Σ_1^0 tale che:

$$f(x_1, \ldots, x_k) = y \iff \mathbb{N} \models \varphi(x_1, \ldots, x_k, y)$$

Da questo lemma il teorema è praticamente fatto, come segue.

Dimostrazione. Se S è semidecidibile, allora esiste f ricorsiva parziale tale che $S = \{f(x_1, \dots, x_k) = 0\}$, dal lemma questo equivale a dire che $S = \{(x_1, \dots, x_k) : \mathbb{N} \models \varphi(x_1, \dots, x_k, 0)\}$, e $\varphi(x_1, \dots, x_k, 0)$ di classe Σ_1^0 per le ipotesi del lemma, da cui S è di classe Σ_1^0, e sono tutte delle equivalenze. $\square$

Dimostriamo ora il lemma.

Dimostrazione. Induzione sulla costruzione di f.

composizione

Se $h = f \circ (g_1, \dots, g_k)$, con $f : \mathbb{N}^k \to \mathbb{N}_\perp$ e $g_i : \mathbb{N}^m \to \mathbb{N}_\perp$ ricorsive parziali, per ipotesi induttiva esistono formule $\varphi_f(\mathbf{z}, y)$ e $\varphi_{g_i}(\mathbf{x}, z_i)$ di classe Σ_1^0 che rappresentano i rispettivi grafi. Allora

$$h(\mathbf{x}) = y \iff \exists z_1 \dots \exists z_k \Big(\bigwedge_{i=1}^{k} \varphi_{g_i}(\mathbf{x}, z_i) \wedge \varphi_f(z_1, \dots, z_k, y) \Big),$$

che è ancora Σ_1^0.

ricorsione primitiva

Sia

$$f(0, \mathbf{x}) = h(\mathbf{x}), \qquad f(y+1, \mathbf{x}) = g\big(y, f(y, \mathbf{x}), \mathbf{x}\big),$$

con h, g ricorsive parziali. Per ipotesi induttiva i grafi di h e g sono Σ_1^0. Usando la codifica β delle sequenze finite, $f(y, \mathbf{x}) = b$ equivale all'esistenza di una sequenza codificata che parte dal valore di $h(\mathbf{x})$, applica il passo dato da g per tutti gli indici $< y$ e termina in b:

$$\exists p \exists q \Big(\varphi_h(\mathbf{x}, \beta(p, q, 0)) \wedge \beta(p, q, y) = b \wedge \forall i < y \, \varphi_g\big(i, \beta(p, q, i), \mathbf{x}, \beta(p, q, i+1)\big) \Big).$$

Per la chiusura di Σ_1^0 rispetto ai quantificatori limitati, otteniamo ancora una formula di classe Σ_1^0.

minimizzazione

Se $f(\mathbf{x}) = \mu t \, g(\mathbf{x}, t)$, con g ricorsiva parziale, per ipotesi induttiva esiste $\varphi_g(\mathbf{x}, t, u)$ di classe Σ_1^0 che rappresenta $g(\mathbf{x}, t) = u$. Allora

$$f(\mathbf{x}) = y \iff \exists u \Big(\varphi_g(\mathbf{x}, y, u) \wedge u = 0 \wedge \forall t < y \, \exists v \, (\varphi_g(\mathbf{x}, t, v) \wedge v \neq 0) \Big),$$

che è Σ_1^0 (di nuovo, grazie ai quantificatori limitati).

$\square$

Lemma 2.14 (Codifica β di Gödel delle sequenze finite)
Sia $\beta(a, b, k) = a \bmod (b \cdot (k+1) + 1)$, dati $y, z_0, \dots, z_y$, esiste a, b tali che:

$$\forall k \leq y \; \beta(a, b, k) = z_k$$

Dimostrazione. Pongo

$$b := (y+1)! \cdot \Big(1 + \sum_{i \le y} z_i\Big).$$

Per ogni $k \le y$ definisco $m_k := b(k+1)+1$. Allora $m_k > z_k$. Inoltre i moduli $m_0, \dots, m_y$ sono a due a due coprimi: se $i < j$ e $d \mid m_i, m_j$, allora

$$d \mid (m_j - m_i) = b(j-i),$$

ma anche $d \mid m_i = b(i+1)+1$, quindi $d \mid 1$ e dunque $d = 1$. Per il teorema cinese del resto esiste a tale che

$$a \equiv z_k \pmod{m_k} \qquad (k \le y).$$

Siccome $0 \le z_k < m_k$, il resto di a modulo m_k è esattamente z_k, quindi

$$\beta(a,b,k) = a \bmod (b(k+1)+1) = z_k \quad \text{per ogni } k \le y.$$

□

Osservazione 2.15 (Complessità di $\beta(a,b,k)$) — La relazione $r = \beta(a,b,k)$ è equivalente a

$$\exists q \, \big(a = q \cdot (b(k+1)+1) + r \wedge r < b(k+1)+1\big),$$

quindi è di classe Σ_1^0 (in effetti anche Δ_0^0).

Abbiamo dimostrato che semidecidibile implica Σ_1^0, vediamo ora l'implicazione inversa.

Lemma 2.16 ($\Delta_0^0 \subseteq \{\text{predicati PR}\}$)
Vale che:

$$\Delta_0^0 \subseteq \{\text{predicati PR}\}$$

Dimostrazione. Induzione strutturale sulle formule di classe Δ_0^0. Le formule atomiche sono uguaglianze tra termini aritmetici, quindi definiscono predicati PR (i termini aritmetici definiscono funzioni PR). La chiusura sotto $\neg, \wedge, \vee$ segue dall'algebra di Boole dei predicati PR. Per i quantificatori limitati, se $P(\mathbf{x}, y)$ è PR, allora anche

$$\exists y \le t \, P(\mathbf{x}, y) \qquad \text{e} \qquad \forall y \le t \, P(\mathbf{x}, y)$$

sono PR, perché si riducono a combinazioni finite di predicati PR su un intervallo limitato. Quindi ogni formula Δ_0^0 definisce un predicato PR. □

A questo punto manca solo la freccia del teorema.

Dimostrazione. Sia S di classe Σ_1^0. Allora esiste una formula $\theta(\mathbf{x}, y)$ di classe Δ_0^0 tale che

$$\mathbf{x} \in S \iff \mathbb{N} \models \exists y \, \theta(\mathbf{x}, y).$$

Per il lemma precedente, esiste un predicato PR $P(\mathbf{x}, y)$ con

$$P(\mathbf{x}, y) = 0 \iff \mathbb{N} \models \theta(\mathbf{x}, y).$$

Definisco la funzione ricorsiva parziale

$$g(\mathbf{x}) := \mu y \, (P(\mathbf{x}, y) = 0).$$

Allora

$$\mathbf{x} \in S \iff g(\mathbf{x}) \downarrow .$$

Per la proposizione di equivalenza tra semidecidibilità e domini di funzioni ricorsive parziali, S è semidecidibile. $\square$

Ammettiamo di avere un'enumerazione delle formule, i.e. $\varphi \mapsto \ulcorner\varphi\urcorner$, con questa enumerazione vale il seguente corollario.

Corollario 2.17 (Teorema di indecidibilità di Church)
L'insieme $\{\ulcorner\varphi\urcorner : \mathbb{N} \models \varphi\}$ non è (semi)-decidibile.

Dimostrazione. Sia

$$V := \{\ulcorner\varphi\urcorner : \mathbb{N} \models \varphi\}.$$

Se per assurdo V fosse decidibile, potremmo decidere H_1, assurdo. Infatti H_1 è semidecidibile, quindi per il teorema precedente esiste una formula $\psi(x, y)$ di classe Σ_1^0 tale che

$$(x, y) \in H_1 \iff \mathbb{N} \models \psi(x, y).$$

Poiché la numerazione di Gödel è effettiva, la mappa

$$(x, y) \mapsto \ulcorner\psi(\overline{x}, \overline{y})\urcorner$$

è ricorsiva totale. Dunque

$$(x, y) \in H_1 \iff \ulcorner\psi(\overline{x}, \overline{y})\urcorner \in V,$$

e questo darebbe una decisione di H_1, contraddizione. $\square$

Morale l'insieme delle verità aritmetiche non è decidibile, e quindi non esiste un'assiomatizzazione ricorsiva delle verità aritmetiche.

§2.2 Teorema di Rosser

Riepiloghiamo gli assiomi della teoria Q di Robinson nel linguaggio $L = L_{\text{arit}} = \{0, +, \cdot, s\}$.

Q1 $\forall x\ \neg s(x) = 0$ (0 non successore);

Q2 $\forall x\ \forall y\ s(x) = s(y) \to x = y$ (iniettività del successore);

Q3 $\forall x\ x + 0 = x$ (def. ricorsiva somma);

Q4 $\forall x\ \forall y\ x + s(y) = s(x + y)$ (def. ricorsiva somma);

Q5 $\forall x\ x \cdot 0 = 0$ (def. ricorsiva prodotto);

Q6 $\forall x\ \forall y\ x \cdot s(y) = (x \cdot y) + x$ (def. ricorsiva prodotto).

Q7 $\forall x\ x = 0 \vee \exists y\ s(y) = x$ (ogni numero eccetto 0 è successore).

Il nostro obiettivo è dimostrare il teorema di Rosser, che sarà una versione più forte del teorema di Gödel.

Teorema 2.18 (Rosser)

Sia T una L-teoria che contiene Q, ovvero $T \vdash \mathsf{Q}$, e sia T **ricorsivamente assiomatizzabile**[a], i.e. l'insieme delle codifiche degli assiomi è ricorsivamente enumerabile, e coerente, allora T è incompleta, ovvero esiste φ chiusa tale che $T \nvdash \varphi$ e $T \nvdash \neg\varphi$.

[a] Si può indebolire a "T **interpreta** Q"

L'idea del perché questa sia la formulazione giusta è che Q ha un numero finito di assiomi, al contrario di PA.

Proposizione 2.19

Sia φ una formula aritmetica chiusa, valgono i seguenti fatti.

- Se φ è di classe Δ_0^0, allora:

$$\mathbb{N} \models \varphi \iff \mathsf{Q} \vdash \varphi \quad \text{e} \quad \mathbb{N} \models \neg\varphi \iff \mathsf{Q} \vdash \neg\varphi$$

- Se φ è di classe Σ_1^0, allora:

$$\mathbb{N} \models \varphi \iff \mathsf{Q} \vdash \varphi$$

Osservazione 2.20 — Le due condizioni sulle formule chiuse in Δ_0^0 implicano che tali formule sono decidibili in Q, infatti essendo chiuse in $\mathbb{N}$ vale necessariamente sempre solo una tra $\mathbb{N} \models \varphi$ e $\mathbb{N} \models \neg\varphi$, e la validità in $\mathbb{N}$ è equivalente alla dimostrabilità in Q.

Dimostrazione. Per formule chiuse Δ_0^0 si procede per induzione strutturale: per il caso atomico si usa il fatto che Q sa fare i conti sui termini chiusi, e per i connettivi/quantificatori limitati si usa che la verifica è finita e che Q dimostra le identità aritmetiche necessarie. Quindi, per Δ_0^0 chiuse, verità in $\mathbb{N}$ e dimostrabilità in Q coincidono sia per φ sia per $\neg\varphi$. Se ora φ è Σ_1^0 chiusa, scrivo $\varphi \equiv \exists x\, \theta(x)$ con $\theta \in \Delta_0^0$. Se $\mathbb{N} \models \varphi$, esiste n con $\mathbb{N} \models \theta(\overline{n})$; per il caso Δ_0^0 ottengo $\mathsf{Q} \vdash \theta(\overline{n})$, quindi $\mathsf{Q} \vdash \exists x\, \theta(x)$. Viceversa, se $\mathsf{Q} \vdash \varphi$, allora $\mathbb{N} \models \varphi$ per correttezza di Q rispetto al modello standard. $\square$

Definizione 2.21 (Numerali)**.** Sia $n \in \mathbb{N}$, chiamo $\overline{n} \overset{\text{def}}{=} s(s(\ldots s(0)\ldots))$, con n successori, **numerale** di n, formalmente questi sono termini di L_{arit} e:

$$\overline{n} := \begin{cases} \overline{0} = 0 & \text{se } n = 0 \\ \overline{n+1} = s(\overline{n}) & \text{altrimenti} \end{cases}$$

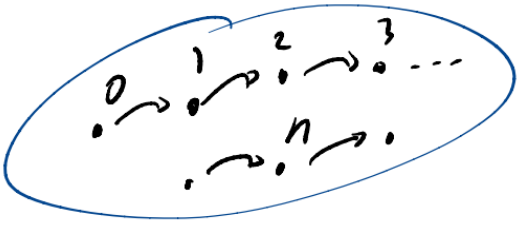
Alternativamente un numerale è una mappa che associa ogni naturale ad un L_{arit}-termine.

Lemma 2.22 (Q sa fare i conti)

Sia t un termine di L_{arit} chiuso, detto $n = \{\ldots\}_{\mathbb{N}} t$, allora vale che:

$$Q \vdash t = \overline{n}$$

ovvero l'interpretazione del numerale coincide con quella del termine t che dà lo stesso numero.



Dimostrazione. Induzione strutturale sugli L_{arit}-termini (chiusi). □

Lemma 2.23 (Caso base del teorema)
Sia φ una formula atomica chiusa (solo un'uguaglianza di termini è possibile), allora:

$$\mathbb{N} \models \varphi \iff \mathsf{Q} \vdash \varphi \quad \text{e} \quad \mathbb{N} \models \neg\varphi \iff \mathsf{Q} \vdash \neg\varphi$$

Definizione 2.24 (Ordinamento sui termini aritmetici)**.** Siano t_1, t_2 L_{arit}-termini, dico che $t_1 \leq t_2$ se $\exists x\ x + t_1 = t_2$.[9]

Definizione 2.25 (Elementi standard e non standard)**.** Sia $M \models \mathsf{Q}$, sia $n \in M$, dico che n è **standard** o **non standard** se $\exists x\ M \models n = \overline{x}$ o $\forall x \in M\ M \models n \neq \overline{x}$.

Si possono pensare come elementi fuori dalla catena dei successori, e andranno pensati come più grandi di tutti i numerali.

Lemma 2.26
$\forall n \in \mathbb{N}$ vale che:

$$\mathsf{Q} \vdash \forall a(a \leq \overline{n} \leftrightarrow (a = \overline{0} \vee a = \overline{1} \vee \ldots \vee a = \overline{n}))$$

ovvero ogni elemento minore o uguale ad un numerale è uguale ad un numerale.

Lemma 2.27
Sia $M \models \mathsf{Q}$, sia $\alpha \in M$ non standard, allora:

$$\forall n \in \mathbb{N}\ M \models \alpha > \overline{n}$$

ovvero ogni elemento non standard è maggiore di ogni numerale.

Finalmente possiamo dimostrare la proposizione che ci serve per il teorema di Rosser.

[9] Questa è una relazione di ordinamento, Q non è in grado di dimostrare che è un ordinamento, inoltre si poteva anche mettere $+x$ a posto di $x+$, visto che Q non è in grado di dimostrare che $+$ è commutativo.

Corollario 2.28
Sia $f : \mathbb{N}^k \to \mathbb{N}$ ricorsiva, allora c'è φ di classe Σ_1^0 tale che:

$$\forall x_1, \dots, x_k \in \mathbb{N}\ \forall y \in \mathbb{N}\ \big(\mathsf{Q} \vdash \varphi(\overline{x_1}, \dots, \overline{x_k}, \overline{y}) \leftrightarrow f(x_1, \dots, x_k) = y\big)$$

ovvero ogni funzione ricorsiva totale è rappresentabile in Q.

Lemma 2.29
Sia $f : \mathbb{N}^k \to \mathbb{N}_\perp$ ricorsiva totale, allora c'è φ di classe Σ_1^0 tale che:

$$\forall x_1, \dots, x_k \in \mathbb{N}\ \forall y \in \mathbb{N}\ \big(\mathsf{Q} \vdash \varphi(\overline{x_1}, \dots, \overline{x_k}, \overline{y}) \leftrightarrow f(x_1, \dots, x_k) = y\big)$$

che differisce dal corollario precedente perché in questo caso la Q non solo dimostra i casi particolari ma proprio il caso generale in cui ho addirittura una funzione degli y che associa y a $f(x_1, \dots, x_k)$.

Dimostrazione. Segue immediatamente dal corollario precedente, perché l'ipotesi di totalità su f è esattamente quella richiesta lì. □

§3 Teoremi di incompletezza

§3.1 Numerazione di Gödel

Una numerazione di Gödel è in generale un modo di associare ad ogni formula di un linguaggio una numero naturale, il modo specifico di farlo non è importante, l'importante è che sia una codifica efficace, ovvero che sia possibile risalire alla formula a partire dal numero e viceversa, e che sia possibile decidere se un numero è la codifica di una formula o meno.

Fissiamo una codifica effettiva dei simboli di L_{arit} mediante numeri naturali distinti e usiamo la codifica di liste $\langle a_1, \dots, a_n \rangle$ introdotta prima per concatenare i codici. Per esempio usiamo tag distinti

$$\#0, \#s, \#+, \#\cdot, \#=, \#\neg, \#\to, \#\forall, \#x.$$

La scelta concreta dei numeri è irrilevante: ciò che conta è che le operazioni sintattiche fondamentali siano funzioni ricorsive.

Definizione 3.1 (Codifica di termini e formule)**.** Definiamo ricorsivamente la codifica di Gödel $\ulcorner \cdot \urcorner$ come segue.

- termini:
$$\ulcorner 0 \urcorner := \langle \#0 \rangle, \qquad \ulcorner x_i \urcorner := \langle \#x, i \rangle,$$
$$\ulcorner s(t) \urcorner := \langle \#s, \ulcorner t \urcorner \rangle, \qquad \ulcorner t_1 + t_2 \urcorner := \langle \#+, \ulcorner t_1 \urcorner, \ulcorner t_2 \urcorner \rangle,$$
$$\ulcorner t_1 \cdot t_2 \urcorner := \langle \#\cdot, \ulcorner t_1 \urcorner, \ulcorner t_2 \urcorner \rangle;$$

- formule:
$$\ulcorner t_1 = t_2 \urcorner := \langle \#=, \ulcorner t_1 \urcorner, \ulcorner t_2 \urcorner \rangle, \qquad \ulcorner \neg\varphi \urcorner := \langle \#\neg, \ulcorner \varphi \urcorner \rangle,$$
$$\ulcorner \varphi \to \psi \urcorner := \langle \#\to, \ulcorner \varphi \urcorner, \ulcorner \psi \urcorner \rangle, \qquad \ulcorner \forall x_i\, \varphi \urcorner := \langle \#\forall, i, \ulcorner \varphi \urcorner \rangle.$$

Proposizione 3.2 (Predicati sintattici ricorsivi)
I seguenti predicati su $\mathbb{N}$ sono PR:

- $\mathsf{Term}(n)$: "n è codice di un termine";
- $\mathsf{For}(n)$: "n è codice di una formula";
- $\mathsf{Sent}(n)$: "n è codice di una formula chiusa";
- $\mathsf{Free}(n, i)$: "x_i è libera nella formula codificata da n".

Dimostrazione. Si procede per ricorsione strutturale sulla forma del codice, distinguendo il tag iniziale tramite $\mathtt{nth}(n, 0)$ e lunghezza tramite $\mathtt{length}(n)$. Tutti i test effettuati sono combinazioni di predicati PR già costruiti su liste e codici, quindi i predicati risultano PR. □

Proposizione 3.3 (Numerali e sostituzione)
Valgono i seguenti fatti.

- La funzione
$$\mathsf{Num}(n) := \ulcorner \overline{n} \urcorner$$
è PR.
- La funzione di sostituzione sintattica
$$\mathsf{Sub}(\ulcorner \varphi \urcorner, i, \ulcorner t \urcorner) := \ulcorner \varphi[t/x_i] \urcorner$$
è PR.

Dimostrazione. Per Num basta la ricorsione:
$$\mathsf{Num}(0) = \ulcorner 0 \urcorner, \qquad \mathsf{Num}(n+1) = \ulcorner s(\overline{n}) \urcorner = \langle \#s, \mathsf{Num}(n) \rangle.$$
Per Sub si ricorre sulla struttura sintattica codificata da n: nel caso di variabile x_j si sostituisce con t solo se $j = i$, e negli altri costruttori si ricompone ricorsivamente sui sotto-codici. □

Lemma 3.4 (Diagonalizzazione di Gödel)
Sia $\varphi(x_i)$ una formula con x_i libera. Esiste una frase chiusa γ tale che:
$$\mathsf{Q} \vdash \gamma \leftrightarrow \varphi[\overline{\ulcorner \gamma \urcorner}/x_i].$$

Dimostrazione. Consideriamo la funzione ricorsiva totale
$$f(n) := \mathsf{Sub}(n, i, \mathsf{Num}(n)).$$
Per il corollario già dimostrato (rappresentabilità in Q delle funzioni ricorsive totali) esiste una formula $\psi_f(u, v)$ di classe Σ_1^0 tale che
$$\mathsf{Q} \vdash \psi_f(\overline{u}, \overline{v}) \leftrightarrow v = f(u).$$

Definiamo

$$\theta(u) := \exists v\,(\psi_f(u,v) \wedge \varphi[v/x_i]).$$

e poniamo

$$\gamma := \theta(\overline{\ulcorner\theta\urcorner}).$$

Allora in Q otteniamo:

$$\gamma \leftrightarrow \theta(\overline{\ulcorner\theta\urcorner}) \leftrightarrow \varphi(\overline{f(\ulcorner\theta\urcorner)}) \leftrightarrow \varphi(\overline{\ulcorner\gamma\urcorner}),$$

perché $f(\ulcorner\theta\urcorner) = \ulcorner\theta(\overline{\ulcorner\theta\urcorner})\urcorner = \ulcorner\gamma\urcorner$. □

§3.2 Predicato di dimostrazione

Definizione 3.5 (Teoria ricorsivamente assiomatizzabile)**.** Una teoria T è ricorsivamente assiomatizzabile se l'insieme

$$\{\ulcorner\sigma\urcorner : \sigma \text{ è un assioma di } T\}$$

è ricorsivamente enumerabile.

Osservazione 3.6 — In particolare, se gli assiomi sono dati da uno schema indicizzato da un predicato semidecidibile, si può passare ad una teoria equivalente ricorsivamente assiomatizzabile esplicitando in modo effettivo le istanze ammesse dallo schema.

Lemma 3.7 (Predicato di dimostrazione)
Sia $T \supseteq \mathsf{Q}$ ricorsivamente assiomatizzabile. Esiste un predicato ricorsivo binario $\mathsf{Dim}(d,n)$ tale che, per ogni formula chiusa σ,

$$T \vdash \sigma \iff \mathbb{N} \models \exists d\,\mathsf{Dim}(d, \ulcorner\sigma\urcorner).$$

Dimostrazione. Un codice d rappresenta una lista finita di formule

$$\langle \delta_0, \ldots, \delta_m \rangle,$$

e $\mathsf{Dim}(d,n)$ dice che:

- ogni riga è o un assioma di T o segue da righe precedenti per modus ponens;
- l'ultima riga ha codice n.

Le verifiche interne usano solo predicati ricorsivi su liste, codici sintattici e regole di inferenza fissate, quindi si ottiene un predicato ricorsivo come richiesto. □

Osservazione 3.8 — La formula $\exists d\,\mathsf{Dim}(d,n)$ è di classe Σ^0_1.

§3.3 Primo teorema di incompletezza

Teorema 3.9 (Primo teorema di incompletezza di Gödel)
Sia T una teoria aritmetica con

$$\mathsf{Q} \subseteq T, \qquad T \text{ ricorsivamente assiomatizzabile}, \qquad \mathbb{N} \models T.$$

Allora T è incompleta.

Dimostrazione. Consideriamo

$$\chi(x) := \neg\exists d\, \mathsf{Dim}(d, x).$$

Per diagonalizzazione esiste una frase G tale che

$$\mathsf{Q} \vdash G \leftrightarrow \chi(\overline{\ulcorner G \urcorner}),$$

dunque anche

$$T \vdash G \leftrightarrow \neg\exists d\, \mathsf{Dim}(d, \ulcorner G \urcorner).$$

Escludiamo i due casi.

- Se $T \vdash G$, allora esiste (metateoricamente) un codice d_0 di una dimostrazione di G, quindi
$$\mathbb{N} \models \mathsf{Dim}(d_0, \ulcorner G \urcorner),$$
e quindi
$$T \vdash \exists d\, \mathsf{Dim}(d, \ulcorner G \urcorner).$$
Dall'equivalenza diagonale segue anche $T \vdash \neg\exists d\, \mathsf{Dim}(d, \ulcorner G \urcorner)$, assurdo.
- Se $T \vdash \neg G$, allora per l'equivalenza diagonale
$$T \vdash \exists d\, \mathsf{Dim}(d, \ulcorner G \urcorner).$$
Poiché $\mathbb{N} \models T$, otteniamo
$$\mathbb{N} \models \exists d\, \mathsf{Dim}(d, \ulcorner G \urcorner),$$
quindi esiste davvero una dimostrazione di G in T, ossia $T \vdash G$, di nuovo assurdo.

Quindi $T \nvdash G$ e $T \nvdash \neg G$. □

Mostriamo ora (in forma compatta) la dimostrazione del teorema di Rosser enunciato sopra.

Dimostrazione del teorema di Rosser. Sia $\mathrm{Neg}(n)$ la funzione ricorsiva che manda il codice di una formula nel codice della sua negazione. Applichiamo il lemma di diagonalizzazione alla formula

$$\vartheta(x) := \forall d \left(\mathsf{Dim}(d, x) \to \exists d' \leq d\, \mathsf{Dim}(d', \mathrm{Neg}(x))\right).$$

Otteniamo una frase R con

$$\mathsf{Q} \vdash R \leftrightarrow \forall d \left(\mathsf{Dim}(d, \ulcorner R \urcorner) \to \exists d' \leq d\, \mathsf{Dim}(d', \ulcorner \neg R \urcorner)\right).$$

Se $T \vdash R$, prendo il primo codice d_0 di dimostrazione di R: dall'equivalenza ottengo una dimostrazione di $\neg R$ con codice $\leq d_0$, contro la minimalità e quindi contro la coerenza.

Se $T \vdash \neg R$, usando la forma negata della stessa equivalenza e il primo codice di dimostrazione di $\neg R$, si ottiene simmetricamente una dimostrazione di R, ancora in contraddizione con la coerenza.

Dunque $T \nvdash R$ e $T \nvdash \neg R$, quindi T è incompleta. □

Proposizione 3.10 (Completamento decidibile)
Sia T una teoria coerente e decidibile. Allora esiste $T' \supseteq T$ coerente, completa e decidibile.

Dimostrazione. Fissiamo un'enumerazione efficace delle frasi chiuse $(\varphi_i)_{i\in\mathbb{N}}$ e definiamo per ricorsione:

$$T_0 := T,$$

$$T_{i+1} := \begin{cases} T_i \cup \{\varphi_i\} & \text{se } T_i \nvdash \neg\varphi_i, \\ T_i \cup \{\neg\varphi_i\} & \text{se } T_i \vdash \neg\varphi_i. \end{cases}$$

Poi poniamo

$$T' := \bigcup_{i\in\mathbb{N}} T_i.$$

Per costruzione T' estende T, resta coerente e decide ogni frase chiusa. Inoltre la procedura è effettiva perché la deducibilità in ogni passo è decidibile. □

Corollario 3.11
Q e PA non sono decidibili.

Dimostrazione. Se una delle due fosse decidibile e coerente, per la proposizione precedente ammetterebbe un'estensione coerente completa e decidibile. Ma ogni estensione coerente ricorsivamente assiomatizzabile di Q è incompleta per Rosser. Contraddizione. □

Corollario 3.12
La teoria vuota nel linguaggio aritmetico non è decidibile.

Dimostrazione. Supponiamo decidibile la teoria vuota (cioè la validità logica in L_{arit}). Poiché Q è finitamente assiomatizzata da $\alpha_1, \ldots, \alpha_m$, per ogni frase chiusa φ vale

$$\mathsf{Q} \vdash \varphi \iff \vdash (\alpha_1 \wedge \cdots \wedge \alpha_m) \to \varphi.$$

Quindi una procedura decisionale per la teoria vuota renderebbe decidibile anche Q, in contraddizione con il corollario precedente. □

Corollario 3.13
L'insieme

$$\{\ulcorner\varphi\urcorner : \mathbb{N} \models \varphi\}$$

non è ricorsivamente enumerabile.

Dimostrazione. È la teoria completa e coerente $\mathrm{Th}(\mathbb{N})$, che estende Q. Se fosse ricorsivamente enumerabile, sarebbe un controesempio al teorema di Rosser. □

§3.4 Secondo teorema di incompletezza

Definizione 3.14 (Provabilità interna in PA)**.** In questa sottosezione fissiamo la formula binaria $\mathsf{Dim}(d, n)$ per $T = \mathsf{PA}$, nel senso che $\mathsf{Dim}(d, n)$ esprime "d è una dimostrazione in PA della formula con codice n". Consideriamo la formula

$$\neg\exists d\, \mathsf{Dim}(d, \ulcorner \bot \urcorner),$$

che esprime la consistenza interna di PA.

Lemma 3.15 (Condizioni di derivabilità)
Per ogni frase φ, ψ valgono:

- se $\mathsf{PA} \vdash \varphi$, allora $\mathsf{PA} \vdash \exists d\, \mathsf{Dim}(d, \ulcorner \varphi \urcorner)$;
- $$\mathsf{PA} \vdash \big(\exists d\, \mathsf{Dim}(d, \ulcorner \varphi \to \psi \urcorner)\big) \to \Big(\big(\exists d\, \mathsf{Dim}(d, \ulcorner \varphi \urcorner)\big) \to \big(\exists d\, \mathsf{Dim}(d, \ulcorner \psi \urcorner)\big)\Big);$$
- $$\mathsf{PA} \vdash \big(\exists d\, \mathsf{Dim}(d, \ulcorner \varphi \urcorner)\big) \to \exists e\, \mathsf{Dim}\big(e, \ulcorner \exists d\, \mathsf{Dim}(d, \ulcorner \varphi \urcorner) \urcorner\big).$$

Teorema 3.16 (Secondo teorema di incompletezza di Gödel)
Se PA è coerente, allora

$$\mathsf{PA} \nvdash \neg\exists d\, \mathsf{Dim}(d, \ulcorner \bot \urcorner).$$

Dimostrazione. In questa dimostrazione poniamo

$$\Box\varphi := \exists d\, \mathsf{Dim}(d, \ulcorner \varphi \urcorner).$$

Sia γ la frase di Gödel per PA:

$$\mathsf{PA} \vdash \gamma \leftrightarrow \neg\Box\gamma.$$

Riscriviamo le condizioni di derivabilità nella forma usata nel diagramma:

- se $\mathsf{PA} \vdash \varphi$, allora $\mathsf{PA} \vdash \Box\varphi$;
- $\mathsf{PA} \vdash \Box\varphi \to \Box\Box\varphi$;
- $\mathsf{PA} \vdash \big(\Box\varphi \wedge \Box(\varphi \to \psi)\big) \to \Box\psi$.

Vogliamo dimostrare

$$\mathsf{PA} \vdash \gamma \leftrightarrow \neg\Box\bot.$$

- $\mathsf{PA} \vdash \gamma \to \neg\Box\bot$.

$$\mathrm{In}_{\to}\,\dfrac{\mathrm{El}_{\to}\,\dfrac{\mathsf{PA}\vdash\big(\Box\bot\wedge\Box(\bot\to\gamma)\big)\to\Box\gamma \qquad \mathrm{In}_{\wedge}\,\dfrac{\mathsf{PA}\vdash\Box\bot \qquad \mathsf{PA}\vdash\Box(\bot\to\gamma)}{\mathsf{PA}\vdash\Box\bot\wedge\Box(\bot\to\gamma)}}{\mathsf{PA}\vdash\Box\gamma}}{\mathsf{PA} \vdash \Box\bot \to \Box\gamma}$$

$$\mathrm{RaA}\,\dfrac{\mathsf{PA} \vdash \Box\bot \to \Box\gamma}{\mathsf{PA} \vdash \neg\Box\gamma \to \neg\Box\bot}.$$

e, usando il ramo $\mathsf{PA} \vdash \gamma \to \neg\Box\gamma$ dell'equivalenza di Gödel,

$$\mathsf{PA} \vdash \gamma \to \neg\Box\bot.$$

- $\mathsf{PA} \vdash \neg\Box\bot \to \gamma$**.** Dall'equivalenza di Gödel basta mostrare $\mathsf{PA} \vdash \neg\Box\bot \to \neg\Box\gamma$, perché poi si usa il ramo $\mathsf{PA} \vdash \neg\Box\gamma \to \gamma$. Il passaggio principale è:

$$\mathrm{In}_{\to}\frac{\mathsf{PA} \vdash \Box\big(\gamma \to (\neg\gamma \to \bot)\big)}{\mathsf{PA} \vdash \Box\gamma \to (\Box\neg\gamma \to \Box\bot)}$$

$$\mathrm{El}_{\to}\frac{(\star)\ \mathsf{PA} \vdash \Box\gamma \to \Box\neg\gamma \qquad \mathsf{PA} \vdash \Box\gamma \to (\Box\neg\gamma \to \Box\bot)}{\mathsf{PA} \vdash \Box\gamma \to \Box\bot}$$

$$\mathrm{RaA}\frac{\mathsf{PA} \vdash \Box\gamma \to \Box\bot}{\mathsf{PA} \vdash \neg\Box\bot \to \neg\Box\gamma}.$$

dunque

$$\mathsf{PA} \vdash \neg\Box\bot \to \gamma.$$

Espandiamo ora il ramo $(\star)$:

$$(\star)\quad \mathrm{In}_{\to}\frac{\mathrm{El}_{\to}\dfrac{\mathsf{PA}\vdash\Box(\Box\gamma\to\neg\gamma) \qquad \mathsf{PA}\vdash\Box\gamma}{\mathsf{PA}\vdash\Box\neg\gamma}}{\mathsf{PA} \vdash \Box\gamma \to \Box\neg\gamma}$$

dove $\mathsf{PA} \vdash \Box(\Box\gamma \to \neg\gamma)$ si ottiene dalla prima condizione di derivabilità applicata a $\mathsf{PA} \vdash \Box\gamma \to \neg\gamma$, e quest'ultima segue dall'equivalenza di Gödel.

Quindi

$$\mathsf{PA} \vdash \gamma \leftrightarrow \neg\Box\bot.$$

Se per assurdo $\mathsf{PA} \vdash \neg\Box\bot$, allora $\mathsf{PA} \vdash \gamma$. Per la prima condizione di derivabilità otteniamo $\mathsf{PA} \vdash \Box\gamma$, mentre dall'equivalenza di Gödel segue anche $\mathsf{PA} \vdash \gamma \to \neg\Box\gamma$ e quindi $\mathsf{PA} \vdash \neg\Box\gamma$. Questo contraddice la coerenza di PA. Dunque

$$\mathsf{PA} \nvdash \neg\Box\bot,$$

cioè

$$\mathsf{PA} \nvdash \neg\exists d\, \mathsf{Dim}(d, \ulcorner\bot\urcorner).$$

$\Box$